UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚĂ ȘI TEHNOLOGIE
POLITEHNICA BUCUREȘTI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL DE CALCULATOARE



PROIECT DE DIPLOMĂ

CityAirQ: Infrastructură backend și model de inteligență artificială pentru monitorizarea calității aerului

Dumitru Bianca-Andreea

Coordonator științific:
Conf.Dr.Ing. Laura Ruse
Conf.Dr.Ing. Dan Tudose
Ing. Abhinuv Pitale

BUCUREȘTI
2025

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE AND ENGINEERING DEPARTMENT





DIPLOMA PROJECT

CityAirQ: Backend Infrastructure and Artificial Intelligence Model for Air Quality Monitoring

Dumitru Bianca-Andreea

Thesis advisor:
Conf.Dr.Ing. Laura Ruse
Conf.Dr.Ing. Dan Tudose
Ing. Abhinuv Pitale

BUCHAREST
2025

CONTENTS

List of figures **5**

List of tables **5**

1 Introduction **1**

1.1 Context 1

1.2 Problem 2

1.3 Proposed Solution 2

1.3.1 Objectives 3

1.3.2 Relevance 4

1.3.3 Contribution 4

1.4 Thesis Structure 4

2 State of art **6**

2.1 Background 6

2.2 Air Monitoring 6

2.3 IoT Data Processing 7

2.4 AI models in Pollution Prediction 8

2.5 Related work 9

3 Architecture and Implementation **12**

3.1 Project Architecture 12

3.2 Backend Requirements 13

3.3 Client-Server Communication 14

3.3.1 Authorization 14

3.3.2 Data Ingestion 15

3.3.3 Data Retrieval 16

3.4 Backend Architecture . . . 18
3.4.1 Architectural style . . . 18
3.4.2 Google pub/sub Message Queue . . . 19
3.4.3 Services . . . 20
3.4.4 Data Storage . . . 22
3.4.5 Data Processing Flow . . . 22

4 Backend Deployment 24
4.1 GKE Cluster . . . 24
4.1.1 GKE Benefits . . . 24
4.1.2 Cluster configuration . . . 24
4.2 Container Orchestration and Microservice Deployment . . . 25
4.2.1 Entry Point and Internal Services Deployment . . . 26
4.2.2 Database Service Deployment . . . 27

5 Backend Testing and Evaluation 28
5.1 Testing Objectives . . . 28
5.2 Testing Types . . . 28
5.3 Infrastructure Observability and Management Tools . . . 29
5.3.1 Prometheus and Kong metrics . . . 29
5.3.2 Loki and Portainer for management . . . 31
5.4 Conclusion of Evaluation . . . 33

6 Interpolation Model for Pollution Prediction 35
6.1 Introduction to the Interpolation Problem . . . 35
6.1.1 Motivation . . . 35
6.1.2 Importance . . . 35
6.1.3 Overview . . . 36
6.2 Problem Definition . . . 36
6.2.1 Formal Statement of the Prediction Task . . . 36

6.2.2 Input Variables 37
6.2.3 Output Values 37
6.3 Data Sources and Structure 38
6.4 Exploratory Data Analysis 39
6.5 Implementation Details 42
6.5.1 Interpolation using IDW 42
6.5.2 XGBoost Regressor 43
6.5.3 Final model: Spatial Interpolation and Correction using XGB 43
6.6 Model Evaluation - Results and Analysis 44

7 Conclusions and Further Work **48**

7.1 Conclusions 48
7.2 Further Work 48

LIST OF FIGURES

1 Deaths by risk factor[1] . . . 2

2 Air Quality Index Categories [2] . . . 8

3 CityAirQ Architecture . . . 12

4 Ingest Request . . . 16

5 Results Request . . . 16

6 Results Response . . . 17

7 High Level Backend Architecture . . . 18

8 Detailed Backend Architecture . . . 20

9 Table's Structure . . . 22

10 Nodes' organization . . . 26

11 Time-series graphs for HTTP requests and CPU consumption Dashboard over a month . . . 30

12 Resource Usage and Request Volume Dashboard over a month . . . 30

13 Kong Overview . . . 31

14 Kong Analytics . . . 31

15 Logging Dashboard . . . 32

16 Portainer Overview . . . 32

17 Cluster's services displayed in Portainer . . . 33

18 Alerts management . . . 33

19 Initial Dataset . . . 38

20 Data distribution . . . 39

21 Daily Patterns . . . 40

22 Seasonal Patterns 41
23 Correlations 42
24 High-level diagram 45
25 Real and predicted values for PM_{10} 46
26 Real and predicted values for $PM_{2.5}$ 46
27 Real and predicted values for NO_2 47

LIST OF TABLES

1 Air Pollutant Factors 7
2 Backend's Services 21
3 GKE vs Self-Hosted Kubernetes for Backend Deployment 25

SINOPSIS

Poluarea aerului este una dintre cele mai urgente probleme de mediu ale societății, având efecte negative grave asupra sănătății umane și asupra mediului. Proiectul CityAirQ propune o soluție eficientă și inovatoare pentru monitorizarea calității aerului, combinând tehnologii IoT și inteligență artificială pentru a analiza și prezice nivelurile de poluare urbană. Contribuția esențială a acestui proiect a constat în dezvoltarea unei infrastructuri backend scalabile, sigure și eficiente, capabilă să gestioneze în timp real date provenite de la senzori mobili montați pe biciclete. Această infrastructură a fost proiectată să susțină cerințele aplicației Android, oferind servicii fiabile de prelucrare, stocare și livrare a datelor. De asemenea, proiectul a inclus implementarea unui model de inteligență artificială, antrenat să estimeze valorile lipsă și să ofere predicții precise privind nivelurile de poluare.

ABSTRACT

Air pollution is one of the most pressing environmental issues in today's society, having severe negative effects on both human health and the environment. The CityAirQ project proposes an efficient and innovative solution for air quality monitoring, combining IoT technologies and artificial intelligence to analyze and predict urban pollution levels. A key contribution of this project was the development of a scalable, secure, and efficient backend infrastructure, capable of handling real-time data collected from mobile sensors mounted on bicycles. This infrastructure was designed to meet the requirements of the Android application, providing reliable services for data processing, storage, and delivery. Additionally, the project included the implementation of an artificial intelligence model trained to estimate missing values and generate accurate predictions of pollution levels.

1 INTRODUCTION

Over the centuries, technological progress has consistently advanced human society. Nonetheless, this progress often came with environmental costs, one of the most persistent being pollution, which attracted the attention of governments and academics worldwide. The most obvious impacts were registered in urban areas, primarily driven by industrial activities, essential for academic growth and a fast expansion of urban populations.

1.1 Context

According to the World Health Organization - WHO[1], air pollution refers to the introduction of harmful substances into the atmosphere, which disrupt its natural balance and can pose health risks to humans and ecosystems alike.Therefore, air quality plays an important role in maintaining climate stability and the ecosystems' health. Unfortunately, as technology and human activity expand, pollution levels are increasing and often aren't controlled as they should be.

Historically, there are traces of air quality analysis back to Ancient Greece. In some writings thought to be made by Hippocrates (around 400 BC), he argued that many diseases were caused by the poor air's quality. He also wrote about people that were living in cities starting to develop deep voices because the air was typically impure and unhealthy [3]. While this evidence of early pollution was found, there wasn't a significant environmental degradation until the **Industrial Revolution**. Factories, heavily reliant on coal as their primary energy source, became major contributors to air pollution. The process of burning coal affected severely the atmosphere because of large amounts of sulfur dioxide (SO_2) and nitrogen oxides (NO_x), which were also highly detrimental to human health. Additionally, because of the polluted environment, acid rains started to form, damaging in this way crops, forests, and buildings, significantly worsening environmental degradation [4].

Since then, humanity has begun to adopt habits and technological strategies to avoid the widespread impact of pollution. We are trying to develop less environmentally aggressive technologies and embrace behaviors and mentalities that support the environment.

[1] https://www.who.int/health-topics/air-pollutiontab=tab$_1$

1.2 Problem

Air pollution is recognized by numerous studies as one of the big risk factors for global mortality. The main diseases are: heart disease, stroke, lower respiratory infections, lung cancer, diabetes, and chronic obstructive pulmonary disease (COPD).

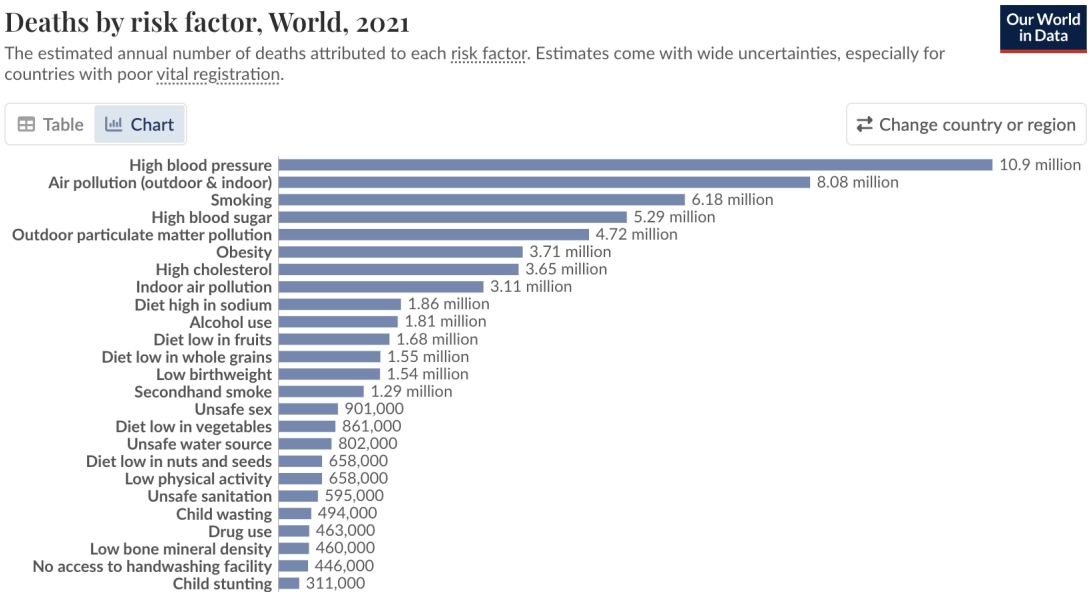


Figure 1: Deaths by risk factor[1]

Data from a study by Our World in Data [1], presented in the Figure 1, shows that air pollution caused death for close to **8.08 million deaths** between 1990 and 2021. To lower this large number of deaths, many efforts and investments focused on creating technologies that are less harmful to the environment. However, less attention has been given to creating tools for mapping the polluted regions. If we had more ways to detect these dangerous places, people and communities could avoid them, which would help reduce how much bad air they breathe.

1.3 Proposed Solution

The CityAirQ project is building an affordable and energy-efficient Internet of Things (IoT) system to track urban pollution. The proposed solution contains pollution detection devices attached to bicycles, turning everyday commutes into a way of collecting and analysing pollution data. Each device is equipped with sensors, including those for detecting main pollutant factors, pressure, temperature, and humidity. The datasets collected are transferred to mobile application and uploaded to our cloud. As a result, the mobile application provides a real-time pollution map, alerts or other communication channels when pollution levels exceed safe thresholds.

1.3.1 Objectives

While the majority of the project's main objectives were achieved within a year, its backend infrastructure, further elaborated in this thesis, was not implemented. I will first provide an overview of these initial achievements, then present the new objectives accomplished in my project:

Accomplished objectives:

- **Hardware device** - A compact prototype was developed, ideal for bicycle integration. This device incorporates sensors for pollutant factors detection, as well as for collecting needed data such as humidity, pressure, and altitude. It will serve as the primary tool for raw data gathering.
- **Mobile Application** - The mobile application will serve as the user interface (UI), providing real-time sensor and historical data visualizations through charts and maps. It incorporates dynamic mapping and location-aware functionalities, alongside a notification service for critical alerts. This application will also function as the exclusive client-side endpoint for the backend infrastructure, facilitating bidirectional communication between the hardware devices and the cloud platform. Its design prioritizes resource efficiency and optimized power consumption.

Backend Objectives:

- **Stream processing** -Given the continuous data ingestion from the hardware, the backend must be engineered to efficiently process **time-series data**. This necessitated the development of a scalable and reliable architecture capable of sustaining high-velocity data streams.
- **Scalable and Efficient Backend Architecture** - Our primary objective is to establish a scalable and efficient backend infrastructure from the beginning, mainly because the project has a high expectancy of developing new functionalities and adding new components such as AI models. A scalable design ensures the backend ability to evolve into a significantly larger system as the project grows.
- **Security** - Similar to any modern application, security is an important project objective. Considering the project's still incipient stage, an important security level was implemented to serve as the foundation for a progressively more advanced security as the project grows.

AI Model Objectives:

- **Spatial Data Computation** - Using spatial interpolation to estimate values for coordinates with missing data.
- **Temporal Prediction** - Applying corrections to the initially interpolated data with a temporal prediction model, incorporating patterns and dependencies observed over time.

- **Maximization of Overall Value Accuracy** - Achieving optimal accuracy in final values by merging spatial interpolation and temporal adaptation techniques.

1.3.2 Relevance

The CityAirQ project offers significant advantages, starting with its innovative design: a sensor device attached to bicycles. Beyond its environmental benefits, this approach encourages cycling as an alternative to public transport and personal vehicles. This is particularly relevant for Bucharest, a city facing severe traffic congestion (ranking 8th worldwide [5]) and high pollution levels (among the top 10 most polluted cities in Europe and 4th in Eastern Europe [6]). Moreover, our project presents a practical way towards improving urban sustainability. The project's devices also offer an important advantage through low energy consumption, achieved by using Bluetooth Low Energy (BLE) coupled with optimized hardware and software. These characteristics are important in differentiating this app from its competitors.

1.3.3 Contribution

My core contributions to the CityAirQ project were the development of its **backend infrastructure** and the **AI model**, both fundamental to the project's success and functionality.

The backend plays a critical role in data management, being responsible for securely storing all incoming sensor data and user-specific information. The backend doesn't just passively hold data, it actively preprocess it. This preprocessing stage is the key for ensuring that the app can execute useful tasks such as Ai model analysis or the vizualization of pollution data for the user.

The AI model is equally important. Its purpose is to predict spatio-temporal pollution data for coordinates lacking direct sensor measurements. This predictive capability is key to overcoming data gaps and ensuring good environmental mapping. Therefore, the AI's ability to provide approximated pollution factors in these unreachable areas is important for the Android application's main feature - a live map displaying pollutant factors within a 10 km radius.

1.4 Thesis Structure

The thesis is organized as follows:

- **Chapter 2 - State of Art:** This chapter provides a large overview of the current air quality monitoring market, examining their general characteristics. It also presents the key advantages that differentiate our proposed project to other existing ones. The research was created based on various articles and documentation, forming a typology

for the other air quality applications, which also marks the conclusion of the project's general overview.

- **Chapter 3 - Architecture & Implementation:** Moving forward to the project's technical aspects, this chapter focuses primarily on the backend architecture and data flow. It details how key design decisions were made and technologies were selected, while also presenting important information about the implemented system design.
- **Chapter 4 - Backend Deployment:** This section details each stage of the backend deployment process, covering the selection criteria for the deployment environment, its benefits, and the process of managing orchestration and subsequent deployment steps.
- **Chapter 5 - Backend Testing and Evaluation:** This chapter describes the technologies used during the testing phase and presents the key findings and conclusions resulted from evaluating the infrastructure's performance metrics.
- **Chapter 6 - Interpolation Model for Pollution Prediction:** This chapter details the development of an effective predictive model that contains an interpolation algorithm. It explores the model's main concepts, the logic behind its implementation, and the processes of creation, testing, and performance analysis.
- **Chapter 7 - Conclusions and Further work:** The final chapter synthesizes the main conclusions that I got from the project's development and execution. Furthermore, it outlines potential future work for expanding the application's performances.

2 STATE OF ART

Before starting any project, it's essential to have a serious research before. This important step helps you clearly understand the project's main purpose, see how others have approached similar subjects, and know what the current market offers.

2.1 Background

Green technology[7] has grown a lot in recent years. Many investors started having over $25 billion into green-tech companies, especially during 2006. This industry started increasing in the late 1990s, and now being at its often called "golden age". It is the third largest sector for investments right after information technology (IT) and biotechnology. This demonstrates how much people are interested in developing green solutions.

One important part of green technology is monitoring air quality. As discussed in earlier parts of this thesis, people started checking air quality in the early 1940. Since then, many tools and apps have been made to help *control air pollution* [8].

The current chapter will further highlight the key concepts for an air monitoring project for pollution.

2.2 Air Monitoring

The main purpose of monitoring air quality is to detect and measure **pollutants** in the atmosphere. By definition, a pollutant is a substance that can contaminate certain regions, whether they are presented in water of air.

Taking this into consideration, there are five main pollutant factors, as presented in Table 1.

The Air Quality Index (AQI) is a standardized indicator used by authorities to inform the public about the current level of air pollution. Its value is calculated from specific pollutant concentrations, using a dedicated formula applied separately for each measured parameter. In practice, the AQI is computed for all pollutants of interest, and the final AQI corresponds to the most severe individual value (sub-index). Each sub-index reflects a linear relationship with the respective pollutant concentration [13].

$$I_p = \left[\frac{I_{HI} - I_{LO}}{B_{PHI} - B_{PLO}}\right] \cdot (C_p - B_{LO}) + I_{LO} \tag{1}$$

Table 1: Air Pollutant Factors

Pollutant factors	Description
Particulate matter (PM2.5 and PM10)	**Particulate matter** is a mix of microscopic solid and liquid substances that can be identified in the air. These particles vary in size, and some can be visible, like smoke or dust, while others are so specialized equipment is required to detect them [9].
Nitrogen dioxide (NO2)	This gas is **poisonous**, and it can be seen when it has contact with fine particulate matter particles, resulting into a reddish-brown color of smoggy air. It is formed by the oxidation of nitric oxide or by combustion in the presence of air [10].
Sulfur dioxide (SO2)	It is **resulted from burning fuel**, such as coal, petroleum oil or diesel [11]. It is composed of sulfur and oxygen and can cause acid rains and respiratory issues, as it was detailed in the Introduction chapter.
Carbon monoxide (CO)	It is usually **resulted from intense traffic**, being produced by an incomplete combustion of carbon-containing fuels [12].
Ozone (O3)	In many cases, it can **cause significant respiratory problems**, at ground level.[9].

- $I_P =$ index of pollutant I
- $C_P =$ truncated concentration of pollutant P
- $B_{PHI} =$concentration breakpoint, greater than or equal to C_P
- $B_{PLO} =$ concentration breakpoint i.e. less than or equal to C_{PI}
- $I_{HI} =$ AQI value corresponding to B_{PHI}
- $I_{LO} =$ AQI value corresponding to B_{PLO}

Considering these aspects, monitoring systems help illustrate both the key pollutant components and the corresponding air quality classification. In many digital platforms, only the AQI is used to determine the pollution level of a given area. Health recommendations are commonly provided , for each AQI range [2], as illustrated in Figure 2.

2.3 IoT Data Processing

In their study on air quality prediction, Claudia Banciu et al. [14] talks about the importance of preprocessing raw sensor inputs to ensure the dataset is accurate and ready for modeling. This involves transforming the initial, potentially noisy and incomplete data into a structured and consistent format using certain data preparation techniques.

This stage is identified by the authors as a fundamental component of the data preparation workflow, often associated with data mining practices. Its main objective is to transform raw

AQI	Category	Health advice
0-33	Very Good	Enjoy activities
34-66	Good	Enjoy activities
67-99	Fair	People unusually sensitive to air pollution: Plan strenuous outdoor activities when air quality is better
100-149	Poor	Sensitive groups: Cut back or reschedule strenuous outdoor activities
150-200	Very Poor	Sensitive groups: Avoid strenuous outdoor activities Everyone: Cut back or reschedule strenuous outdoor activities
200+	Hazardous	Sensitive groups: Avoid all outdoor physical activities Everyone: Significantly cut back on outdoor physical activities

Figure 2: Air Quality Index Categories [2]

and unstructured information into a clear and understandable format. The process involves several structured steps:

- **Data Cleansing**: It is the process of correcting errors or inconsistencies from the raw, initial dataset. It should consist of the following steps: duplicate entries, handling outliers (extreme values for certain data points), and rectifying any data entry errors that may have occurred during collection.
- **Handling Missing Values**: Most of the times, datasets can be inconsistent, having muliple gaps that need to be adressed. The authors note the use of methods like *dropna(inplace = True)*, which removes entire rows containing any missing data.
- **Smoothing Noisy Data**: This process aims to get rid of random errors or "noise" that can occur within the dataset, ensuring the data's integrity.
- **Resolving Inconsistencies**: This step focuses on ensuring uniformity across different data points, which is important for accurate analysis.
- **Converting Data Types**: Numerical entries are converted into appropriate types, such as floating-point numbers, to enhance computational accuracy and ensure compatibility with machine learning algorithms.

These data processing steps are not just technical formalities; they are the foundation of reliable and accurate air quality prediction.

2.4 AI models in Pollution Prediction

Artificial Intelligence (AI) and Machine Learning (ML) models play a crucial role in ensuring accurate and efficient air quality monitoring and prediction, as highlighted in recent research on AI-driven environmental monitoring systems[15]. The article talks about a variety of AI models have been used for this purpose:

- **Recurrent Neural Networks (RNNs) - Long Short-Term Memory (LSTM)**: Frequently used for time series prediction in air quality forecasting, LSTM models utilize

input, output, and forget gates to control information flow and prevent the vanishing gradient problem. They have shown promise in predicting nitrogen dioxide (NO_2) concentrations

- **Gated Recurrent Units (GRU)**: Also a type of RNN that has shown promise in predicting NO_2 concentrations.
- **Hybrid Time-Series Models - Bayesian-optimized hybrid time-series**: This model combines Seasonal Autoregressive Integrated Moving Average (SARIMA) and LSTM. It is usually used in forecasts to increase accuracy of Air Quality Index (AQI). All types of data is used within the model, wether they are linear or non linear.
- **Model Output Statistics (MOS) Approaches - Random Forest (RF)**: This machine learning method creates several decision trees that work together to make predictions. It has been used to adjust regional air quality forecasts and gave the best results when analyzing the link between air pollution and respiratory illnesses.
- **Gradient Boosting**: Used to improve the detail and accuracy of regional air quality predictions.

Other AI Models:

- **Linear Regression**: A method that finds patterns between inputs and outputs, helping to predict future values by looking at past data.
- **Genetic Algorithm (GA)**:Used to find connections between air quality and weather conditions.
- **Genetic Algorithm-based Kernel Extreme Learning Machine (GA-KELM)**: This improved model can predict different pollutants and the air quality index more quickly and accurately than older models like CNNs and RNNs.

In the last decade, deep learning has been used more and more to predict air pollution levels. One newer approach uses encoder-decoder models with attention, which helps the system focus on the most important parts of the data. This method can even deal with missing information by using special networks called adversarial models. Another system, called AIrSense, uses AI to spot strange values and clean up data from low-cost sensors, making the pollution readings more accurate and reliable.

2.5 Related work

After an analysis of the market, **Airly**[1] is the most relevant product, having certain similarities with our project. It was founded by a polish company and it has been operating for an year. Airly provides maps for real-time data, displaying them within a map. Therefore, there are many key aspects about this product that are worth noting:

[1] https://airly.org/map/en/

- **A dense system** - The application provides: hardware devices, web Android and iOS platforms, and air pollution forecasts.
- **Small devices with a fixed placement** - The air quality sensors are fixed in certain locations and can capture information about variations of particulate matter ($PM_{1.0}$, $PM_{2.5}$, PM_{10}) and weather information, such as temperature, pressure, and humidity. Additionally, they have several other methods for gathering data, for example they capture data from Environmental Monitoring station, when their sensors are not accessible to specific locations.
- **AI predictions based on interpolation** - As detailed at the previous point, they use different ways for gathering data. An important one is using algorithms and AI techniques, so they can fill in the gaps between sensors with high accuracy. Similar to the CityAirQ application, they use interpolation algorithms and display the complete dataset, by using heatmaps. The intensity of the color around each sensor shows how confident the system is in its estimate for that area.

Airly is also actively engaged in adapting its project for international markets, collaborating with local authorities to implement their system in various cities.

Similar to Airly, **IQAir**[2] is an air monitoring application, known for its indoor and outdoor products for gathering air-quality data. Moreover, it contains a global data platform named IQAir AirVisual. A few aspects are worth noting for this application:

- **Globally expanded network**: IQAir has an extensive expansion, allowing data collection at high resolution, because it provides one of the largest networks for air monitoring. Their sensors are strategically located in regions such as North America, Europe, East Asia, and Oceania.
- **Analysis and Prediction**: The IQAir AirVisual platform utilizes advanced algorithms for prerpocessing and interpreting collected data.
- **Versatile devices:** IQAir produces various devices, including air monitors, air purifiers, filters, face masks etc. These are designed to be easily integrated into private homes and various urban infrastructures.

PurpleAir[3], an American company, has become a known name in the big scale of the air monitoring field.Their sensor network is considered to be the largest low-cost network globally, with over 22.000 PM sensors in over 100 countries (started in 2023). Therefore, these are the main aspects about PurpleAir:

- **Dense low-cost sensor network:** PurpleAir's has as a main principle providing accessible sensors for everyone, which has led to a noticeable network expansion. This density ensures a good map visualization of air quality, even in regions not covered by governmental stations.

[2]https://www.iqair.com/?srsltid=AfmBOoppW6Qa8St-KhWLY$_B7Xm9EyRgCN9Wmsiv6cVIld0phhCERjRmw$
[3]https://www2.purpleair.com/?srsltid=AfmBOoq90HhISUpjYFye7HRTnWn7qRt2kJDDlLqzA21DIzvAAqQWkBWW

- **Community Involvement and Open Data:** The application differentiates itself by other products because of its way of collecting data. The sensors are available to the general public, allowing anyone to contribute to the global map. This approach increases public awareness and provides valuable data for researchers.
- **Integration with existing platforms:** The platform's data is usually integrated into other air monitoring platforms, including AirNow al EPA from America. This highlights the utility of the information collected by their sensor network, even though they are low-cost.

These examples, demonstrate the market's tendency through air monitoring solutions, more and more dense, precise and accessible. These solutions are built on different technologies for sensors and intelligent algorithms for analysis and data prediction. CityAirQ aligns with this trend, aiming to offer an innovative and efficient solution within this competitive environment.

3 ARCHITECTURE AND IMPLEMENTATION

The chapter dedicated to architecture and implementation plays a key role in understanding how the application's infrastructure is built and works. This chapter details the backend components, how they communicate with each other, as well as the technologies used for processing, storing, and transmitting sensor data. It also explains the architectural decisions made throughout the development of the project.

3.1 Project Architecture

The CityAirQ project consists of 3 important components (Figure 7): hardware devices, mobile application and backend infrastructure.

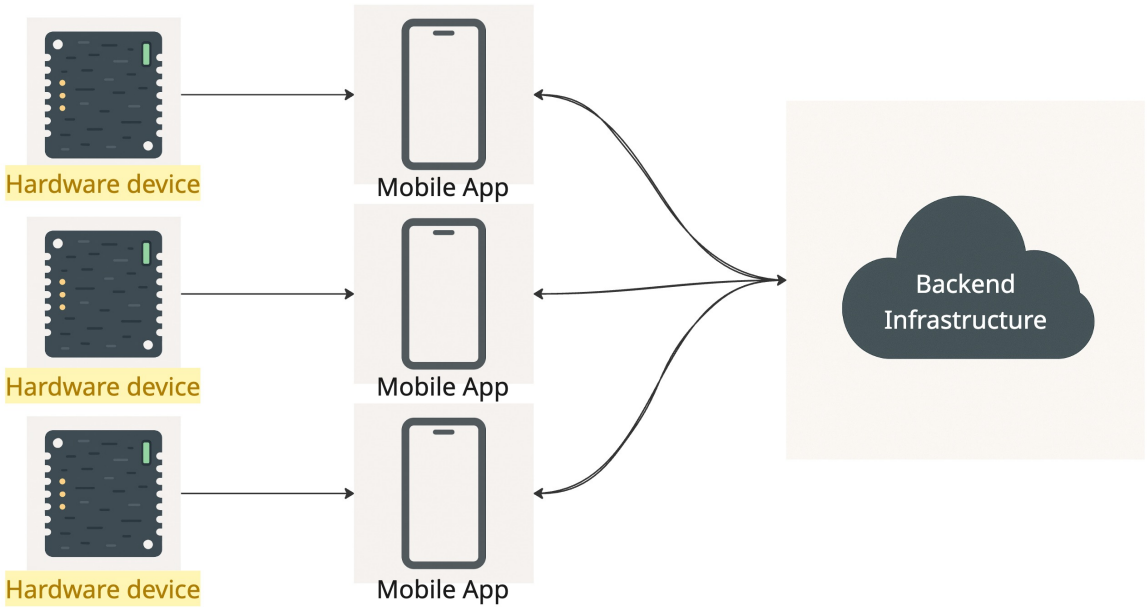


Figure 3: CityAirQ Architecture

The main flow has the following steps:

- **Collecting data using hardware devices attached to bikes:** Portable devices based on the ESP32 microcontroller are equipped with air quality monitoring sensors to capture the needed pollutant factors. These devices communicate with the mobile app via *BLE* (Bluetooth Low Energy), sending environmental data along with the user's current location.

- **Displaying the data in the Android app interface:** The mobile application provides key features for users, including real-time data visualization, charts, pollution warnings, and an interactive map. Additionally, the app sends raw sensor data to the cloud infrastructure and receives preprocessed and refined data to display personalized insights to the user.
- **Processing and analyzing the data in the cloud:** Once received by the cloud infrastructure, the data is preprocessed and stored in a time-series database. The backend architecture includes a custom API designed to the project's needs. Furthermore, the collected data is used to train an AI model that predicts missing pollution values for unmeasured coordinate points, ensuring dataset consistency and spatial coverage.

3.2 Backend Requirements

The architecture had to contain specific key functionalities when it was designed. Taking into consideration the implementation stage when I started my work (the hardware devices were prototyped and the mobile application implemented) I had to create an architecture based on the already implemented components. Furthermore, the whole data-flow was defined by the type of data used in this project, specifically time-series data. The speed and velocity of time series ingestion and the high amount of data in storage need a time series-specific solution[16]. Therefore, I have tried to ensure from the beginning that the infrastructure has the "must-have" characteristics for a time-series architecture and a basic one:

- **Interoperability** - It is the seamless data exchange between applications, devices, and products[16]. It is an important characteristic, because the more the project increases its expansion the more data will need to be transferred. Therefore, problems about latency, speed of response, efficiency could occur.
- **Scalability** - The capacity of an application or system, whether hardware or software, to maintain its performance and reliability as it adapts to changes in size, volume, or demand to meet user needs[17]. Any application must meet this condition of being scalable; it's a necessity for all applications on the market.
- **High Throughput** - Refers of how many units of information a system can process in a given amount of time. High throughput is critical for time-series data because of the continuous stream of incoming data (inserts) and the frequent need to query large volumes of historical data (fetches).
- **Security** - It refers to protecting systems from unauthorized access, preventing cyber threats. [18].

3.3 Client-Server Communication

The backend communicates only with the Android application, which is its only client. The communication pattern used was a Request - Response one, commonly used where a client sends a request to the server and waits for a response [19]. To be more specific, a **HTTP Request-Response protocol** was implemented, which involves one microservice which can submit an HTTP request to another microservice and wait for a response by using HTTP protocols such as REST or SOAP[20]. This protocol was chosen for its ability to provide a rapid, straightforward, and seamless integration within my infrastructure.

In addition, the communication within the infrastructure is divided into two categories:

- **Unidirectional Communication** - the client is not waiting for the server's feedback while it sends the raw data gathered from the sensors. This asynchronous communication ensures a favorable environment for a fast and continuous data ingestion.
- **Bidirectional Communication** - useful for fetching and sending the required data for the client.

3.3.1 Authorization

To ensure a robust level of security and access control within my infrastructure, I implemented an authorization system reliant on **Google OAuth**[1]. This approach allowed us verify the identity of each package which arrived to our endpoint, having a guarantee that only the authorized entities could interact with it.

OAuth 2.0[2], which stands for "Open Authorization", is an authorization protocol and not an authentication protocol. It is designed primarly for granting access to a set of resources, for example, remote APIs or user data. It uses **Access tokens** with expiration time which are piece of data that represents the authorization to access resources on behalf of the end-user.

The protocol is consisted of **roles** which defines the essential components of the OAuth2.0 system:

- **Resource Owner:** The entity that owns the protected resources and has the authority to grant access to them, in this context, the *system administrator*.
- **Client:** The client is the part of the system that needs permission to use protected resources. It must have a valid Access Token to gain access. In our setup, this role is fulfilled by the *mobile application*.
- **Authorization Server:** The Authorization Server is responsible for handling requests from the Client to get Access Tokens. These tokens are provided only after the Resource

[1] https://developers.google.com/identity/protocols/oauth2
[2] https://auth0.com/intro-to-iam/what-is-oauth-2

Owner gives permission. This server offers two main endpoints: one for handling user sign-in and approval (the Authorization endpoint), and another for securely exchanging tokens without user interaction (the Token endpoint). In our case, the Authorization Server is the *Google Auth Platform*.

- **Resource Server:** A server that stores user data and handles access requests sent by the client. It only shares the requested information if the Access Token is valid. In our case, this role is handled by the *backend*.

Given my decision to deploy within the Google Cloud environment, I opted for *Google Auth Platform* which uses OAuth2.0 protocol, from the Google Cloud Console to maintain technological consistency. Also, the platform stores and manages all the service accounts registered as clients, which has access to valid Access Token. The clients are manually registered, an easy process, considering the fact that the only client is the mobile application.

For *Granting access*,in our case, the following steps are required :

1. The Client authenticates as a GoogleSignInClient, having its credentials already registered, and requests an Access Token from the Google Auth Platform.
2. The Json Web Token (JWT) received is included in the Header of the package, as "X-Auth-Token".
3. Once a package reaches the public endpoint, its token validity is verified via a request to the Google Auth Platform. If access is granted, the backend responds with all necessary data.

3.3.2 Data Ingestion

This section was designed for managing a continuous and unilateral data streaming flux from the mobile application. It is optimized for high throughput and low latency, allowing real-time data collection.

We used HTTP POST protocol, so the client can send the packages without waiting for a confirmation, or a specific feedback. Also, the raw data from the sensors were transferred using JSON format, as you can see in the Figure 4.

Therefore, the ingest requests contains the next fields:

- **ClientId:** an id for verifying which user is ingesting data.
- **Timestamp:** a specific field for time-series packages - the data is partitioned based on the timestamp, indicating the exact time of collection; the format is Unix Timestamp.
- **Longitude and latitude:** user's coordinates
- **Data:** the array stores the captured pollutant factors values, which must include data for the following essential fields: : Temperature, Humidity, Pressure, Altitude, CO_2, $PM_{1.0}$, $PM_{2.5}$, PM_{10}.

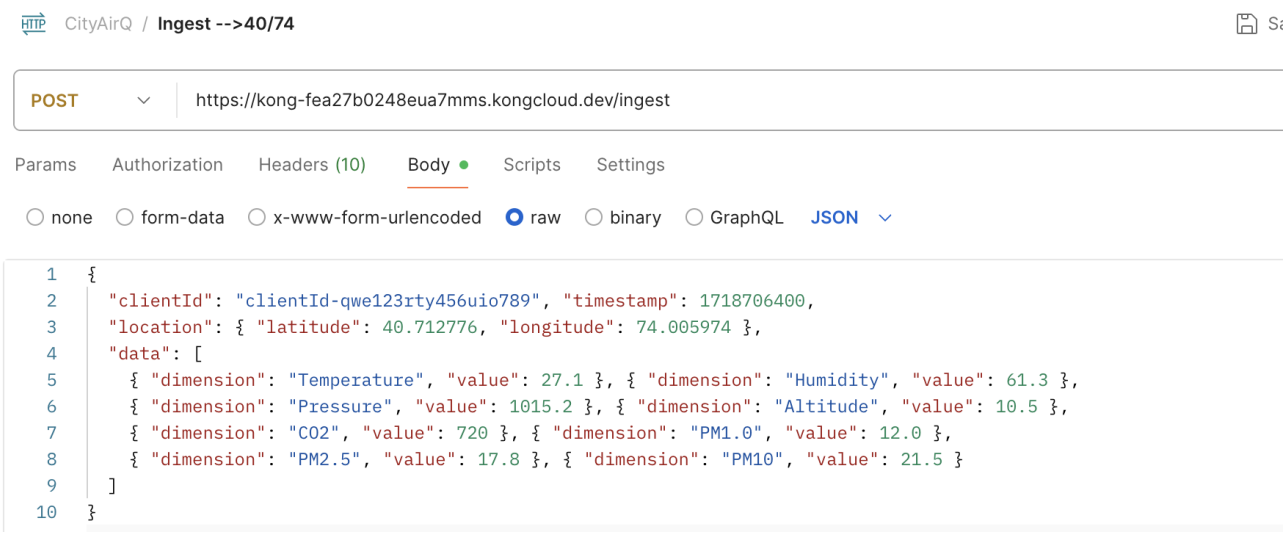


Figure 4: Ingest Request

3.3.3 Data Retrieval

This part of the flow involves a bilateral communication and serves for the mobile application's ongoing requests for displaying real-time pollutant factors on the map, working in parallel with the ingestion processes. Upon receiving a retrieval request, the backend does a series of data preprocessing and adaptation operations. This includes filtering and aggregating the raw data based on the user's location and the most recent timestamp interval.

To maintain consistency in the use of technologies, for retrieval the packages are in JSON format, using HTTP. While a GET request might seem like the most intuitive choice, I opted for a POST method due to the need to include the request details in a JSON body, Figure 5.

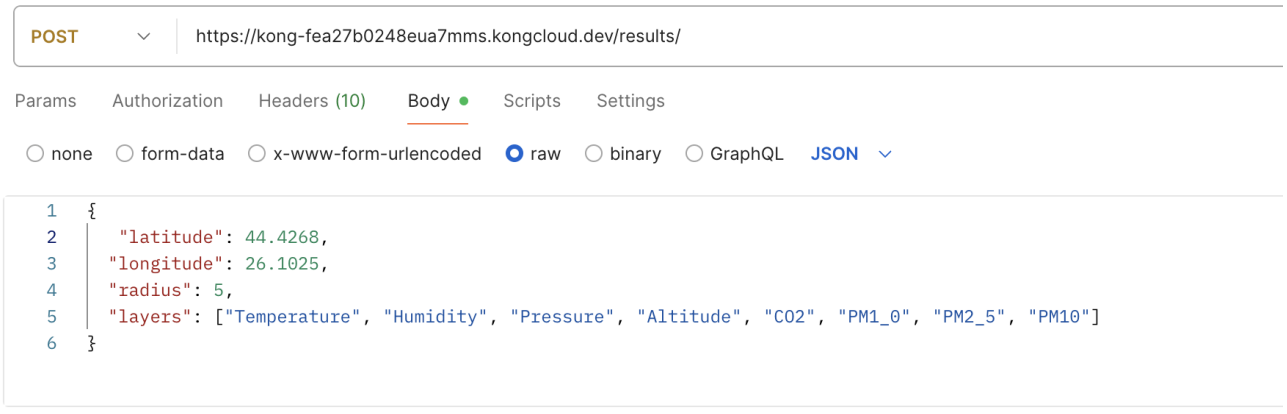


Figure 5: Results Request

The client needs to include in the package the user's coordinates (*latitude* and *longitude*), the radius in kilometers for displaying pollutants within the maximum spatial area requested by the user, and a list of factors to be retrieved.

The backend responds with a JSON package containing a "data" array, where each entry

includes the following fields in the order shown in Figure 6:

- **Client ID** (for identification)
- **Timestamp** (format ISO 8601)
- **Coordinates** (latitude and longitude)
- **Values of Pollutant factors**

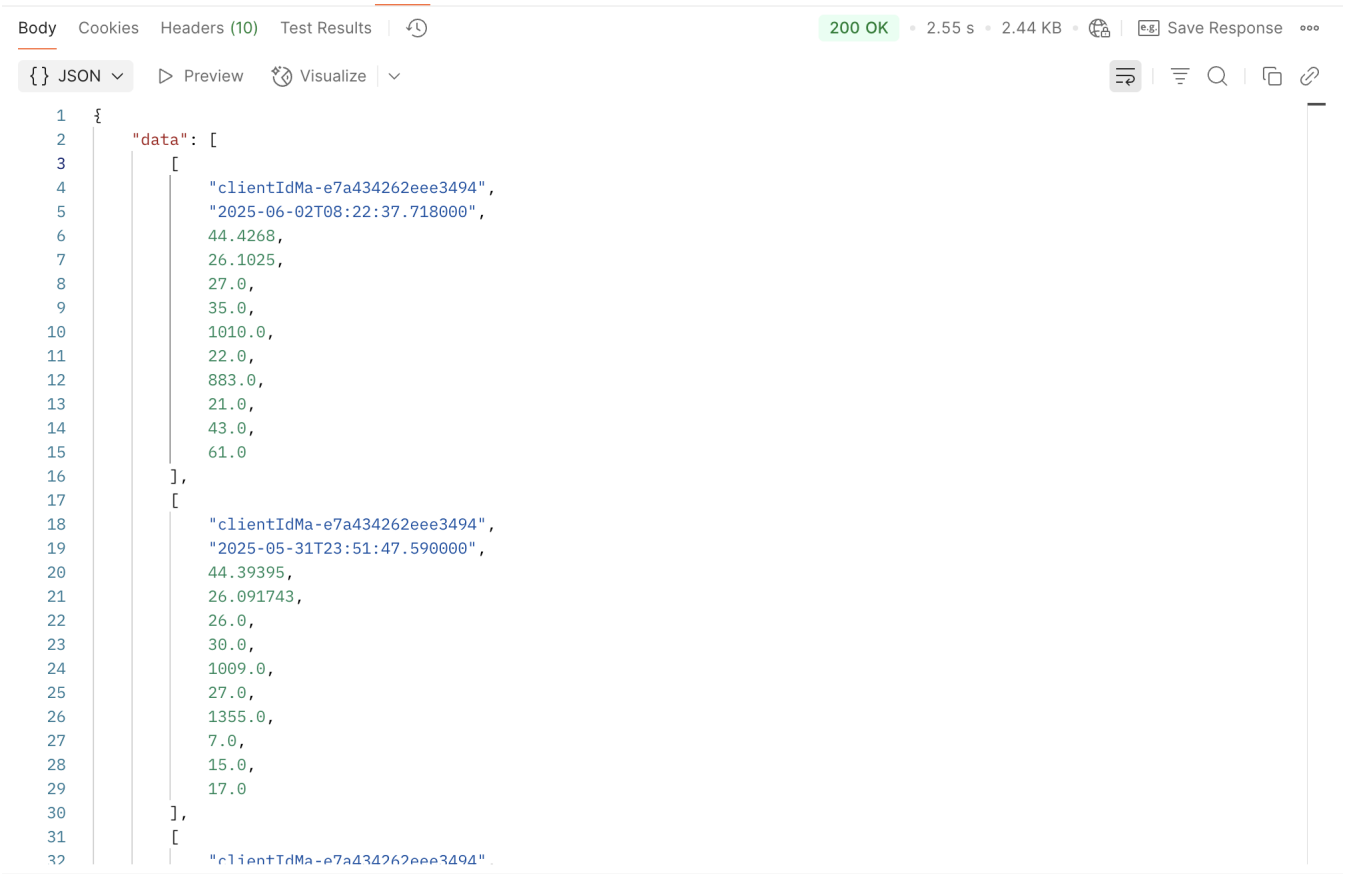


Figure 6: Results Response

3.4 Backend Architecture

The architecture is designed to ensure a scalable, secure, and asynchronous data flow for information sent by the mobile client. As shown in Figure 7, It integrates the Google Auth service for authorization, a public endpoint, and a Google Pub/Sub message queue, which enables the decoupling of components and the parallel processing of messages by internal services.

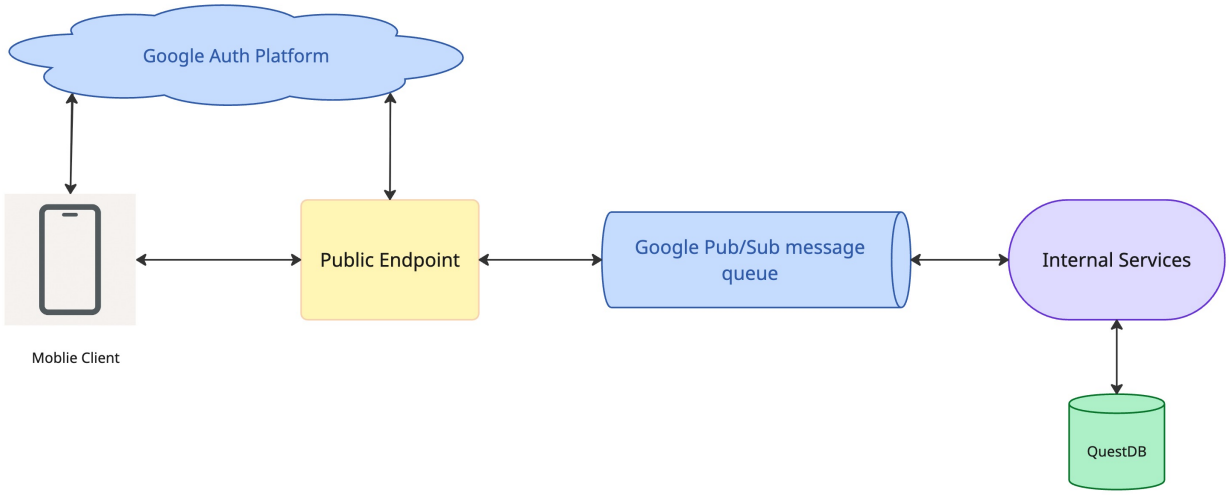


Figure 7: High Level Backend Architecture

3.4.1 Architectural style

The backend's architecture follows a RESTful style, based on standard HTTP protocols and methods.

To clarify my choice I will enumerate the 3 main benefits of a REST API [21]:

1. **Lightweight**
   A REST API relies on a HTTP standard, which means it has an agnostic format, allowing the use of JSON, XML, HTML etc. It makes the architecture fast and lightweight, which are necessities for a structure involving IoT or mobile applications.
2. **Independent**
   One of the most important characteristics that led me to choose this architectural style is its components' independence. My design required integrated time-series data storage and also I wanted it afford the flexibility to incorporate various technologies. This also gave me the opportunity to work on different areas of my project and test different technologies for each component.
3. **Scalable and flexible**

Due to its straightforward implementation and its independence, a REST architecture is easy scalable. As the project is in an incipient stage and more components will be added over time, and REST facilitates the addition of new components with less effort.

As an alternative for the architectural style I considered using Remote Procedure Call (RPC). The choice of this approach was motivated by the characteristics of the application, as well as the scalability requirements, interoperability and simplicity of maintenance.

In the context of the application, REST proved to be a more suitable solution than RPC due to its resource-oriented and clearer structure, which allows a clear separation between the POST methods used in client-server communication and the GET methods used for retrieving performance metrics (more details in the deployment chapter).

3.4.2 Google pub/sub Message Queue

Pub/Sub[3] is a cloud-based messaging system provided by Google Cloud that enables decoupled communication between components by separating message publishers from subscribers.This model supports scalable and flexible interactions between services and is particularly useful in scenarios requiring asynchronous communication, low latency, and high throughput. The Google Cloud messaging system operates asynchronously, ensuring dependable performance and the ability to scale while maintaining minimal response times.

From a usability standpoint, Google Pub/Sub offers a clean and intuitive interface for message handling. It implements a classic publish-subscribe communication model. This communication between publishers and subscribers is achieved through event broadcasting. This design allows the queue to have multiple topics, depending on the type of messages to be transferred or the communication's purpose, with subscribers subscribing to relevant topics and publishers simultaneously publishing their information.

This pipeline was integrated into the infrastructure for the following reasons:

- **Ensuring Continuous Message Intake and Processing Flexibility:**
  The project's architecture is data streaming oriented, which required an asynchronous method for processing messages with low latency. The message queue's design offers the capability for the endpoint to continuously receive messages, while internal services process the data at their own pace. This approach helps to avoid backlog accumulation, increased latency, and data loss, issues that can occur when data processing cannot keep pace with the high rate of incoming data.

- **Providing an easily scalable infrastructure:**
  Given the future AI integration into the architecture, the project required an easy way to work with potential additional services. By using the Google Pub/Sub message

[3]https://cloud.google.com/pubsub/docs/overview

queue, I ensured the seamless addition of any needed internal services without further implementation at the endpoint; all additional services will simply need to subscribe to the topic designated for publishing their required data.

- **Maximized Processing Throughput:**
  Pub/Sub utilizes a unique approach called "per-message" parallelism. It doesn't use partitions for horizontal scaling (message segmentations that must be processed sequentially), unlike many other messaging systems. Instead, messages can be processed independently and simultaneously by each subscriber or instances of the same subscriber. Afterwards, the message queue keeps track of whether the messages were successfully processed or not. This approach maximizes the parallelism of the subscribed services and ensures independence between the publishers and subscribers.

3.4.3 Services

The services within the project's architecture were implemented using a REST API, due to the benefits listed in the previous section of this chapter and the simplicity of deployment explained in the next one. Each service has specific functionalities and is also designed to be flexible, allowing the addition of new features in case the project is expanded.

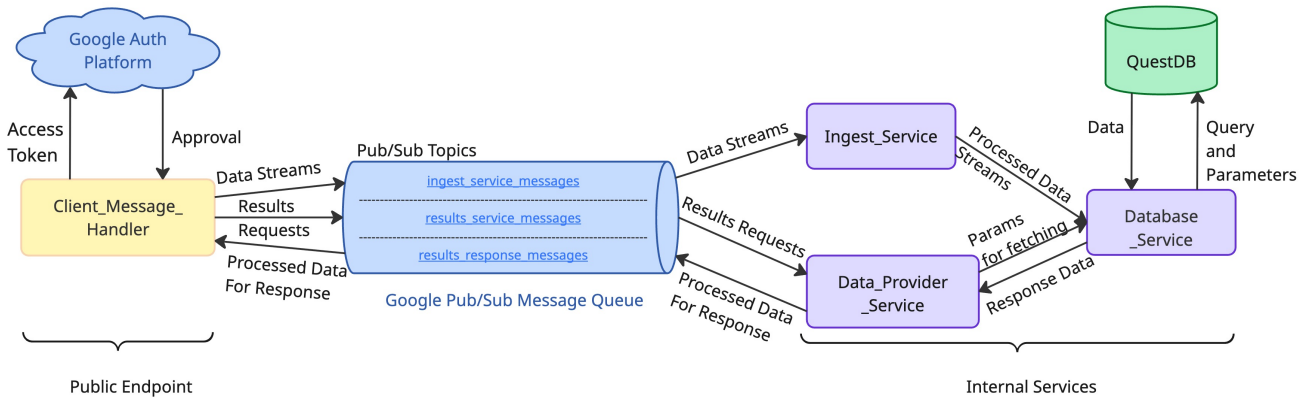


Figure 8: Detailed Backend Architecture

Communication between services is asynchronous, as shown in Figure 8. The Database_Service is the only component that exposes a private endpoint, implemented as a client accessible internally by the other two internal services. The remaining services do not require internal client communication, because the messaging queue system already handles all needed interactions. In the final step of the pipeline, the Database_Service communicates to the Data Storage using PostgreSQL. Furthermore, internally, the packages are transferred in a JSON format.

Each service is isolated in terms of logic and functionality, as shown in Table 2, allowing updates to the code or replacement of specific components without affecting the rest of the system.

Table 2: Backend's Services

Service's Name	Description	Associated Technologies
Client_Message_Handler	Represents the backend public endpoint, whose main responsibilities are to communicate with clients and authorize incoming data packages. This service is also responsible for distributing tasks to internal services, acting as the main publisher in the message queue system.	Google Auth Platform, Google Pub/Sub Message Queue
Ingest_Service	Internal service that handles the data streaming process and prepares the data for ingestion into the Data Storage. It is also subscribed to the topic where the Client_Message_Handler service publishes ingestion requests that need to be processed.	Google Pub/Sub Message Queue, Database_Service
Data_Provider_Service	Internal service whose responsibilities include processing fetching requests from the client, preparing the response data, and publishing the required responses. Similar to the Ingest_Service, this service is also subscribed to a topic where the Client_Message_Handler service publishes fetching requests.	Google Pub/Sub Message Queue, Database_Service
Database_Service	It represents the only connection between the backend and the Data Storage. Its main functionalities are: ensuring connection sessions with the QuestDB, executing SQL commands based on the received requests, providing a client endpoint that can be accessed by the other internal services.	Ingest_Service, Data_Provider_Service, QuestDB, PostgreSQL

3.4.4 Data Storage

In terms of storage, it was opted to a time-series database. **QuestDB** is an open-source solution, optimized to handle continuous data streaming. In addition, its indexing model and internal structure allow real time ingestion with an ultra-low latency, multi-tier storage engine and high ingestion throughput, essential aspects for a monitoring application.

QuestDB includes an integrated Web Console, which makes it even easier to maintain a constant overview of the state of your tables. To ensure fast ingestion and querying, all data received from the sensors is stored using their associated timestamp. The timestamp column serves as the primary reference for ordering, storing, indexing, and optimizing time-based queries. Additionally, the dataset is partitioned by the day of ingestion, which allows faster data retrieval based on specific time intervals. The table's structure is similar to that of the packages received in ingest requests, with columns corresponding exactly to the fields sent by the client.

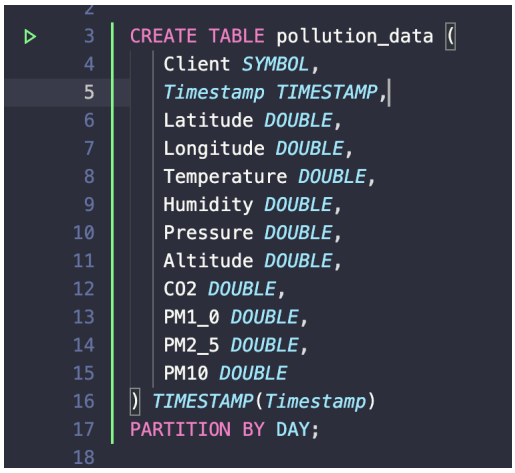


Figure 9: Table's Structure

Furthermore, QuestDB offers full support for SQL, allowing efficient queries without the need for a specific query language, as is the case with some other time-series databases. To establish the connection between the backend and the database, the PostgreSQL client library was used. During the initialization process, a database connection pool is created, which is used for parallel connections and queries to the database, ensuring asynchronous communication.

3.4.5 Data Processing Flow

Currently, this architecture provides 2 main flows: ingestion and retrieval. I will describe all the stages of the flows for a clear overview of the infrastructure, also shown in the Figure 8.

In the init phase, following actions occur simultaneously:

- The services involved in the messaging queue system connect to the Google Pub/Sub service.
- The services subscribe to the corresponding topics and start listening continuously in a separate thread for incoming tasks.
- A pool with database's sessions of connection is initialized.
- The public endpoint establishes its connection to the Google Auth Platform.

The stages for the ingestion flux are as follows:

1. The package from the client arrives at the Client_Message_Handler service through the provided HTTP endpoint for ingestion.
2. The package goes through the authorization process.
3. If the package is authorized, it is published to the message queue on the *ingest_service_messages* topic.
4. Due to its continuous listening, the Ingest_Service immediately receives the package and prepares the data for ingestion into the database.
5. Using the private endpoint exposed by the Database_Service, the package is forwarded to this service.
6. The Database_Service builds the required query for data ingestion.
7. The Database_Service connects to QuestDB using one of the sessions from the connection pool and executes the query.

Similar to the ingestion flux, the retrieval one has the following steps:

1. The package from the client arrives at the Client_Message_Handler service through the provided HTTP endpoint for results.
2. The package goes through the authorization process.
3. If the package is authorized, it is published to the message queue on the *results_service_messages* topic.
4. Due to its continuous listening, the Data_Provider_Service immediately receives the package and prepares the request for fetching.
5. Using the private endpoint exposed by the Database_Service, the package is forwarded to this service.
6. The Database_Service builds the required query for data retrieval.
7. The Database_Service connects to QuestDB using one of the sessions from the connection pool and fetch the required data.
8. The Database_Service is forwarding the array of values to the Data_Provider_Service.
9. The Data_Provider_Service is encapsulating the values into the responding package and publishes it on the *results_response_messages* topic.
10. Due to its continuous listening in a separate thread, the Client_Message_Handler immediately receives the package and sends it to the client.

4 BACKEND DEPLOYMENT

The backend's deployment is a crucial phase in the application development process, ensuring the transfer of software components from a local development environment to a production-ready one accessible to end-users. Within this chapter, I will further detail the required steps to deploy and configure the application's services.

4.1 GKE Cluster

Google Kubernetes Engine (GKE)[1] is a container management service offered by Google Cloud that simplifies the deployment and coordination of containerized applications using Kubernetes[2] is an open-source system designed to run and coordinate container-based applications across various environments. Google Cloud is the "birthplace" of Kubernetes, succeeding in making it publicly accessible in 2014.

Kubernetes, which forms the core of GKE, groups containers into logical units, named "pods" and runs them in administered clusters, providing essential services like load balancing, service discovery, auto-scaling, rolling updates, and self-healing (automatically restarts failed containers). GKE builds on Kubernetes by tightly integrating with Google Cloud, streamlining how compute, network, and storage resources are allocated and monitored.

4.1.1 GKE Benefits

One of the most important benefits of GKE is its natural integration with other Google Cloud services, maintaining consistency across the entire architecture. Additionally, the Google Kubernetes Engine has many benefits compared to self-hosted Kubernetes, as is shown Table 3.

4.1.2 Cluster configuration

A key configuration aspect was choosing the control mode of the cluster. It was chosen a cluster in Standard mode, instead of Autopilot, due to its extended flexibility in managing the cluster. This mode is providing complete control over pods configurations and resource allocations, essential aspects for running a QuestDB instance in one of the containers. Furthermore, in addition to the existing Google Cloud monitoring tools for cluster, new tools

[1] https://cloud.google.com/learn/what-is-kubernetes?hl=en
[2] https://kubernetes.io/docs/concepts/overview/

Table 3: GKE vs Self-Hosted Kubernetes for Backend Deployment

Functionality	**Self-hosted Kubernetes**	**GKE**
Installation and configuration	Requires manual setup	Automated by Google
Maintenance	Done by the user	Automatic updates and patches managed by Google
Monitoring and logging	Requires external tools and integration	Built-in support through Cloud Monitoring and Logging
Scaling	Requires manual configuration	Integrated autoscaling for both nodes and pods
Security	Fully accomplished by the user	Native integration with IAM and other private networks
Implementation time	High setup and configuration time	Significantly reduced deployment time

were required for monitoring the deployed services. These tools required full control over the persistent volumes. Also, to add an API Gateway, it was needed full control over the endpoint's IP address, features that are not accessible in the AutoPilot mode.

The GKE cluster is a zonal one, located in europe-west1-b region, consisting of 3 nodes, with a total of 6 vCPU and 12 GB RAM memory. The choice of the region was motivated by the goal of reducing latency, considering that the majority of machines are located in Europe. The europe-west1 zone provides an optimal trade-off between costs, performance and the geographical proximity to users. The 3 nodes configuration is enough for running all the internal services, including an instance of QuestDB, which needs dedicated resources for proccessing and storing the time-series data.

These configurations provide a balanced combination between flexibility, operational control, and scalability.

4.2 Container Orchestration and Microservice Deployment

The development pipeline is consisting of packaging the application into containers using Docker, and managing its deployment across the cluster with Kubernetes.The first step was to containerize each service individually using Dockerfiles. After defining the components of the system, a docker-compose file was created to launch the complete architecture locally in Docker. This allowed testing the communication between the components, verifying that the ports were correctly exposed, and making sure that the overall architecture works well with an isolated environment, without causing any costs.

To integrate the application with the Google Cloud infrastructure and allow the containers

to run in GKE, the Docker images needed to be uploaded to the *Google Artifact Registry*[3]. An architecture compatibility issue was encountered during this process, because the images were built locally on arm64, while the GKE cluster uses linux/amd64. To resolve this problem, all the images were built using 'docker buildx bake' command and pushed with a flag that explicitly set the target platform to match the cluster's operating system.

After all the images were uploaded in Google Artifact Registry, deployment files were created for each service to define the number of container's replicas, the exposed ports, the required resources and the type of service to configure, resulting in a node organization as shown in Figure 10. These deployment configurations were applied within the cluster, enabling the distribution of services across nodes and an efficient allocation of resources.

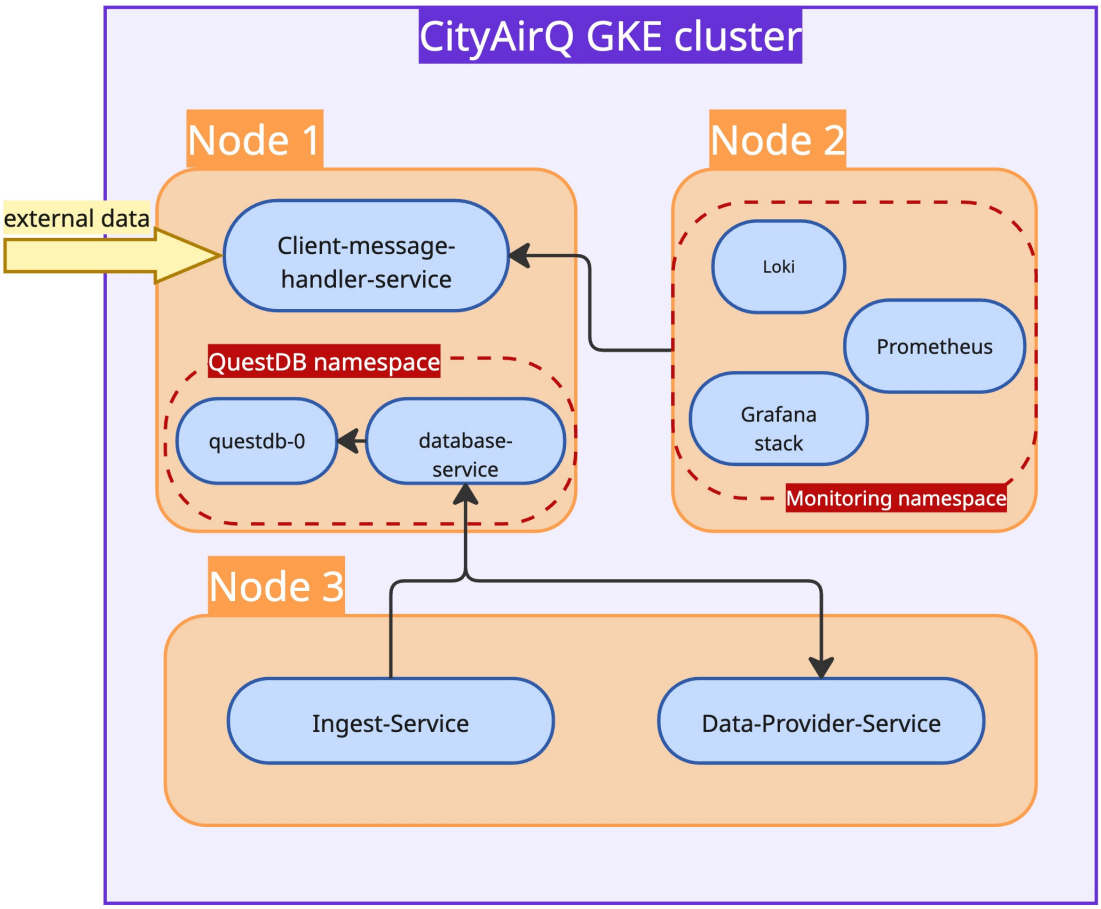


Figure 10: Nodes' organization

4.2.1 Entry Point and Internal Services Deployment

As detailed in the previous chapters, the client-message-handler is the public pod in the architecture. Therefore, it required a TCP port exposed to clients. This was achieved using a Load

[3]https://cloud.google.com/artifact-registry/docs/overview

Balancer service[4], a cluster ingress mechanism available when using a Cloud Provider. When the deployment file is applied to the cluster, the client-message-handler service is assigned a static external ip address, which remains unchanged as long as the cluster and the service are running.

Considering the newly integrated monitoring tools which ensure continuous insight into architecture's health, an additional port was needed for scraping metrics. To support this, an extra port was configured at the entry point, dedicated only for gathering metrics about the traffic data.

To enable integration between the deployment process and the Google Cloud technologies used in the architecture, defining secret files within the Kubernetes deployment was mandatory. Most of Google technologies require authentication via a service account configured in Google IAM[5]. This account holds the necessary credentials for interacting with the corresponding Google platforms. Therefore, secret files were created to store these credentials securely for each service, enabling internal access to resources such as: the Google Pub/Sub message queue and the authorization endpoint provided by Google Auth Platform.

4.2.2 Database Service Deployment

In the deployment process the database required a slightly more effort compared to the other services. In addition to the internal service that communicates directly with the database, an instance of QuestDB needed to be deployed on one of the nodes. Due to its native integration with Kubernetes, this step was straightforward, being installed using Helm[6].

After the instance was deployed, a Load Balancer service was required to expose a port for communication with the other internal services. As a result, new configurations were added to the database-service, which connects to the exposed port of QuestDB and facilitates data exchange with both the ingest-service and the data-provider-service.

[4]https://kubernetes.io/docs/concepts/services-networking/
[5]https://cloud.google.com/security/products/iam
[6]https://helm.sh/

5 BACKEND TESTING AND EVALUATION

5.1 Testing Objectives

The main purpose of the testing process was to validate a proper functioning of all components, and a correctness of the data sent to the client, as well as the seamless integration with the Google Cloud Platform. It was also necessary to verify the overall workflow and the successful deployment of the system.

Considering these aspects, the objectives established for the testing and evaluation process are:

- **Functional verification of each component** - Ensure it performs its assigned role (e.g. ingestion, data processing, data transfer to the front-end).
- **Testing end-to-end flux** - Covering the entire flow, starting with data collecting using the ESP32 device and ending with displaying the information in the mobile application, including communication within the Pub/Sub message queue and backend processing.
- **Data validation** - Verifying wether the data provided by the sensors is correctly stored in the database and retrieved in the expected format.
- **Google Cloud Integration** - Ensuring that the Google Cloud technologies used in the CityAirQ project are correctly integrated, and that data is transferred properly. This includes validating the workflow of the authorization system, the stability of communication with Pub/Sub, and the correctness of the connections with Google Artifact Registry and QuestDB.

5.2 Testing Types

In this section, there are presented the used types of tests in the project, along with the methodologies and tools applied for a proper validation of the project's workflow and performance evaluation after deployment.

The following tests were executed for accomplishing the testing objectives, explained in the previous section:

- **Unit testing**
  What was specifically tested:
  1. functions for data processing and validation;

2. the correct data insertion in the database, by mocking the database instance;
3. client's requests processing and the responses' format;
4. verifying the behaviour in case of invalid input or data missing;
5. correct handling of the authorization token;

For unit testing, the following tools were used: pytest[1] and unittest.mock[2].

- **Integration testing**
  It was essential for validating a correct communication between the components. The objectives of this testing were:

  1. the publishers are correctly publishing the packages in the message queue and the subscribers are receiving the data;
  2. the internal services are correctly communicating with the database-service which has to connect properly to the QuestDB instance;
  3. all the services are connecting to the Google Cloud Technologies, using the secret files in Kubernetes;
  4. the end-to-end flow is working correctly;

  For integration testing, Docker Compose [3] was used for a local testing in an isolated environment.
- **Security and authorization testing:**
  In this process of testing, the main objective was to ensure that only authorized clients could access the backend functionalities. This was possible, by achieved using the OAuth2.0 Playground [4], a platform that can simulate the Google Auth system and provides access tokens for testing purposes.

5.3 Infrastructure Observability and Management Tools

5.3.1 Prometheus and Kong metrics

The monitoring namespace within the GKE cluster, where all monitoring and management tools were installed, ensures a complete visibility over the application's behavior. To evaluate the backend, I implemented a monitoring system based on Prometheus and Kong.

Prometheus[5] is an open-source system for monitoring and alerting, ideal for containerized environments and distributed services. It periodically scrapes services that expose a "/metrics" endpoint, aggregating data such as resource usage, HTTP request counts, CPU consumption, and other performance indicators critical to ensure scalability and reliability.

[1] https://docs.pytest.org/en/stable/
[2] https://docs.python.org/3/library/unittest.mock.html
[3] https://docs.docker.com/compose/
[4] https://developers.google.com/oauthplayground/?code=4
[5] https://prometheus.io/docs/introduction/overview/

All metrics scraped from the exposed endpoint are presented in dashboards, using Grafana stack.

In Figure 11 we can observe the evolution of the number of HTTP requests and CPU usage during the current month. As shown, the number of GET requests increases linearly over time, which is expected, as these requests are generated by Prometheus itself to scrape metrics. In contrast, the POST requests, graph clearly highlights the periods during which the backend was actively used by the mobile application. Regarding CPU usage, the data shows consistent and low overall usage, with occasional spikes reaching up to 0.014s, but without dangerous deviations. This indicates an efficient and stable behavior of the application in terms of processing.

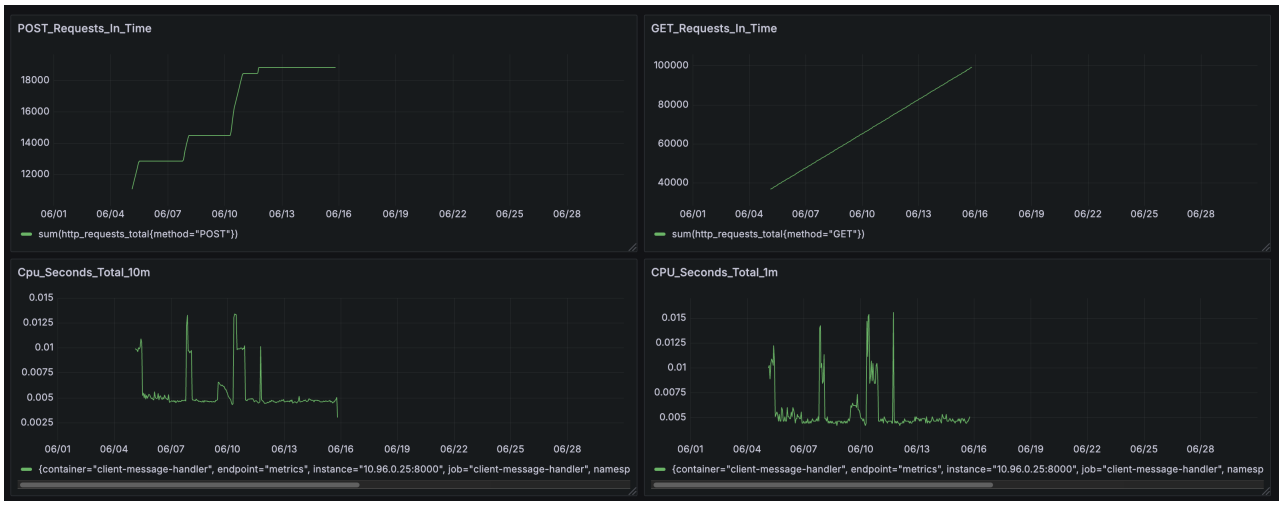


Figure 11: Time-series graphs for HTTP requests and CPU consumption Dashboard over a month

From Figure 11 we can also observe the exact number of HTTP requests received by the endpoint, with a higher number of GET requests than POST, as expected due to Prometheus's continuous metric scraping. Additionally, the resource usage remains stable, with no unusual spikes. This level of stability is essential in containerized environments, where uncontrolled memory consumption can lead to unplanned restarts or availability issues.



Figure 12: Resource Usage and Request Volume Dashboard over a month

Kong Gateway[6] is used as an API Gateway in the application's architecture, as shown in Figure 13. At the same time, it also provides detailed metrics about the HTTP traffic, but for a small interval of time (the current week), comparing to Prometheus.

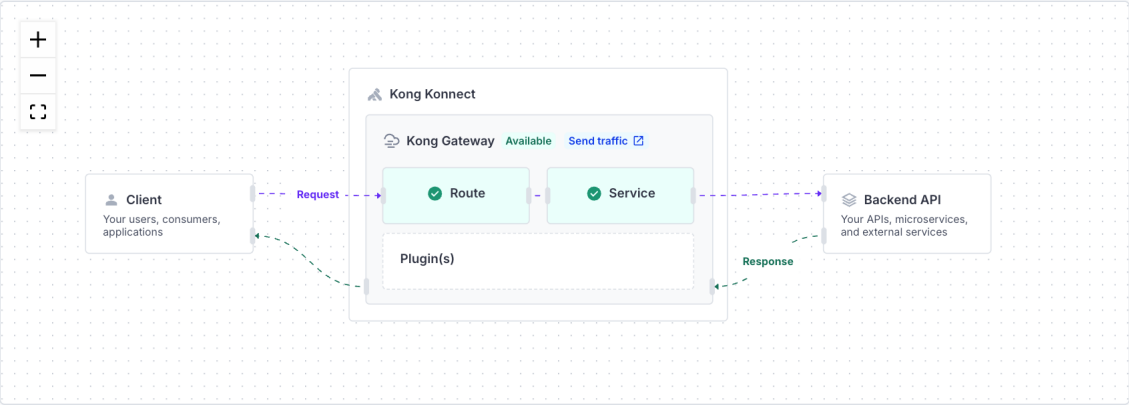


Figure 13: Kong Overview

Kong exposes several types of analytics, including the number of HTTP requests per status code, average latency and globally error rate. As shown in Figure 14, the backend experienced a 41.79% decrease in error rate compared to the previous week,which clearly indicates an improvement in stability. Moreover, although the average latency remains close to 1 second, the decreasing trend is a positive sign. At the same time, a constant latency above 900ms can indicate opportunities for optimization (e.g. backend performance improvements, network optimization, or implementing caching mechanisms).

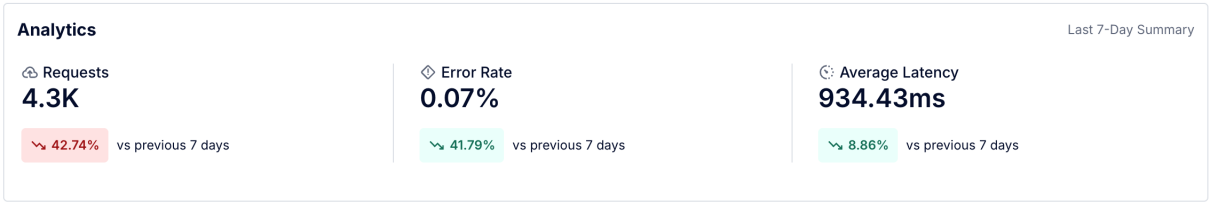


Figure 14: Kong Analytics

5.3.2 Loki and Portainer for management

Besides the monitoring of performance metrics and HTTP traffic, realised by Prometheus and Kong, an efficient administration of the application within a kubernetes environment,also requires tools for logging and resource visualization across the cluster. To achieve this, two additional components were integrated into the GKE setup: Loki, for log collection and analysis, and Portainer, for visual infrastructure management.

Loki is a log aggregation and querying system, developed by Grafana Labs. Unlike to other

[6]https://docs.konghq.com/konnect/

logging systems, Loki is designed to integrate seamlessly with Prometheus and Grafana, having a similar approach by indexing only the labels and not the full content of log messages.

In the current setup, Loki collects logs from the main services (client-message-handler, data-provider-service, and database-service) and displays them within Grafana using a customized dashboard, as shown in Figure 15.

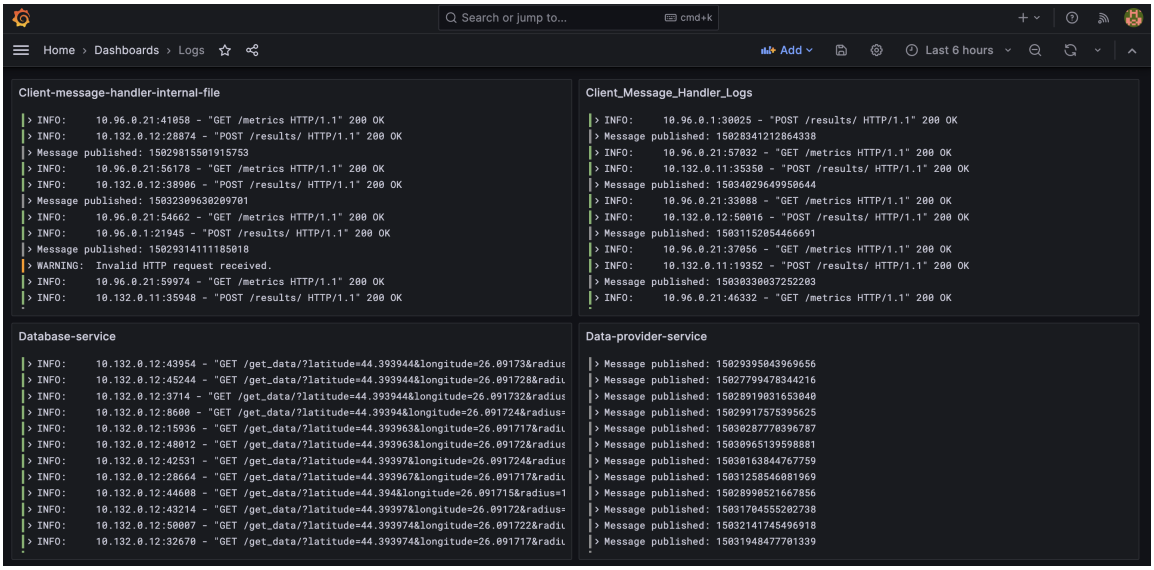


Figure 15: Logging Dashboard

Portainer[7] is a universal container management platform. In the current setup, a Kubernetes environment, Portainer provides a centralized overview of the system components, such as namespaces, services, applications, ConfigMaps, volumes, as well as alerts or node-related events (Figure 16).

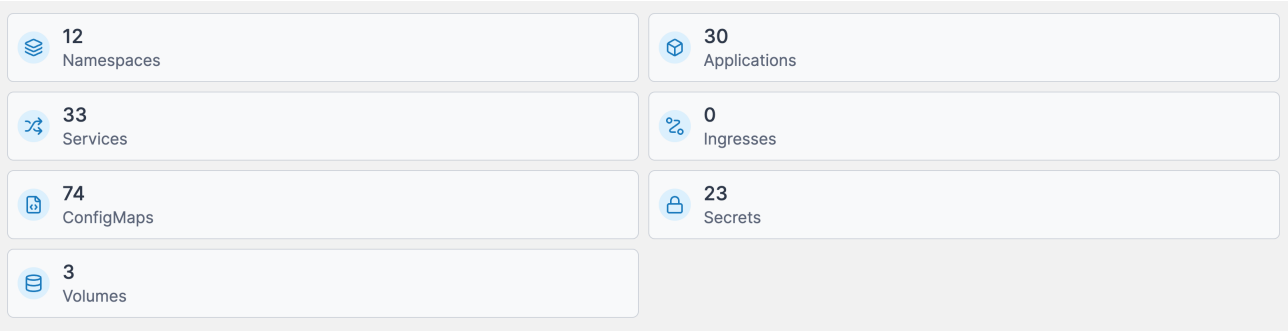


Figure 16: Portainer Overview

Using portainer, managing the cluster became easier, due to its intuitive access to each individual service, Figure 17, providing information about them. Additionally, it includes an integrated terminal, which is also highly useful.

Also for a better visibility and awareness of every single warning and alert, the integrated UI provides a notification system that displays all events occurring in the cluster, Figure 18.

[7]https://www.portainer.io/

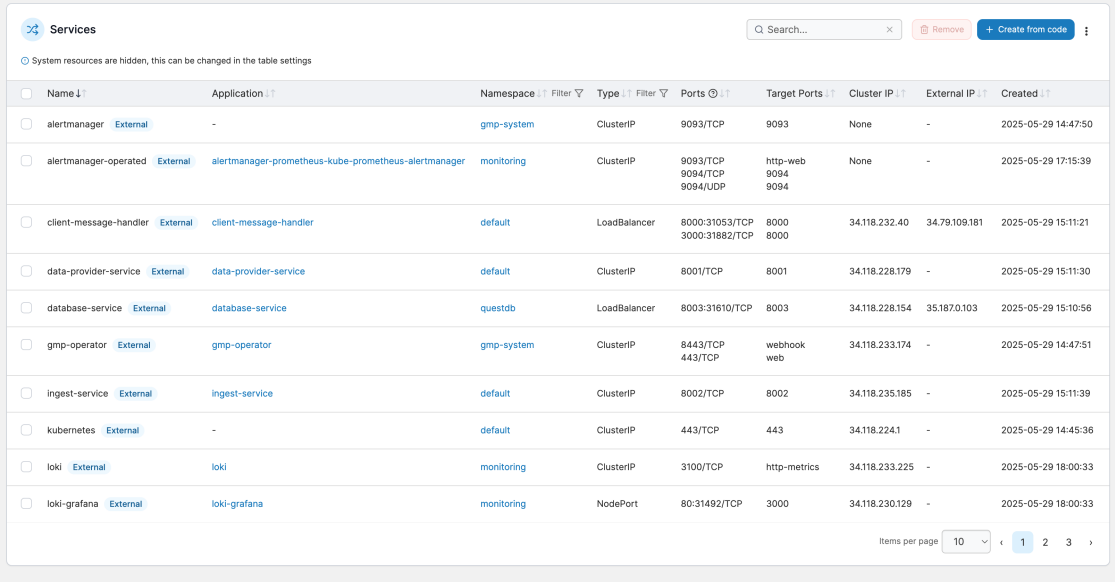


Figure 17: Cluster's services displayed in Portainer

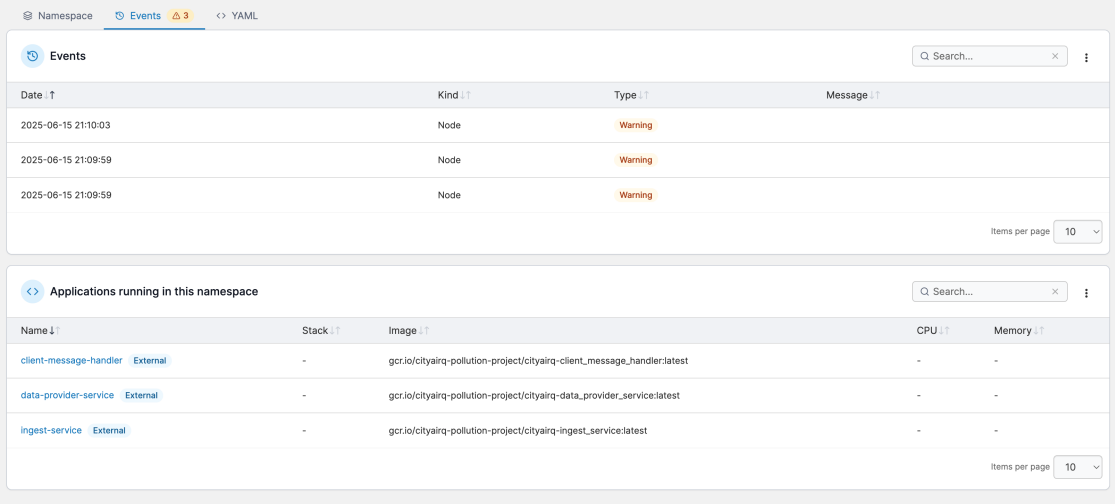


Figure 18: Alerts management

5.4 Conclusion of Evaluation

As a conclusion to the testing process, the correct functioning of all the backend components was successfully validated, as well as the seamless integration within the infrastructure provided by Google Cloud Platform.

The results of the unit and integration tests, demonstrated effective communication between components, as well as good security, using the OAuth2.0 protocol.

In terms of observability and performance, the integration using Prometheus, Grafana and Kong provided a complete overview of the infrastructure. The analysis of the monitoring graphs indicated low and consistent resource consumption, as well as a decreasing error rate comparing to the previous weeks. Although, the average latency remains close to 1 sec,

the decreasing trend is a positive indicator, and also values around 900ms suggest future opportunities for optimization.

The additional components, Loki and Portainer had strengthened the process of administration and vizualization of the infrastructure. By continuously analyzing the logs, it was concluded that the internal services are functioning properly, with occasional errors occurring that were quickly resolved. Since the GKE cluster was configured, no node-related errors have occurred.

In conclusion, the testing phase confirmed that the architecture is reliable and well integrated. The proposed solution has demonstrated stability and scalability, making it suitable for adaptation in real-world production scenarios.

6 INTERPOLATION MODEL FOR POLLUTION PREDICTION

6.1 Introduction to the Interpolation Problem

In the context of the current project, which has air quality monitoring as one of its main objectives, an issue occurs when we want to provide for our users a complete map of pollutant factors captured within its area. Since data is collected only in the locations where the devices physically pass,many areas on the displayed map lack values.

To fill in these gaps and provide a comprehensive and coherent pollution map, we chose to use an interpolation algorithm integrated with a prediction model. The interpolation model has the role to estimate the missing values for the measured pollutants in the unscanned areas, based on the nearby gathered values. Additionally, to improve the accuracy of the estimations and to consider the temporal fluctuations (throughout the day, week, etc.), the output of the interpolation algorithm is further adjusted, using a supervised machine learning model.

In the mathematical field, Interpolation is a method of constructing or finding new points based on the range of known data placed in their proximity.

6.1.1 Motivation

The main motivation for developing this model is the necessity to provide users with a complete and relevant experience when viewing the pollution map in the mobile application. Users expect to see all the pollutant points within their area, not just those directly measured by our devices. It is also a mandatory aspect especially for this early stage of the project, where the amount of collected data is limited and insufficient to fully cover the Bucharest area. Additionally, having a complete dataset allows further data analysis for the prediction model.

6.1.2 Importance

The interpolation of missing data not only improves the user experience, but it is also essential for the long term achievements of the application: constant monitoring, map expansion, and potential support for decision-making. A complete pollution map can be used, for example, to identify critical areas, estimate health risks, and to provide key recommendations for low-pollution routes for cyclists. Moreover, as the project continues to develop, there is strong potential for extending the project beyond the Bucharest area, into other regions. Without this model, the applications would remain limited to areas directly covered by sensors, considerably

reducing the project's overall purpose and utility.

6.1.3 Overview

The model developed in this project is a hybrid approach, combining classical spatial interpolation methods (e.g. Inverse Distance Weighting [22]) with a prediction model, specifically XGBRegressor, to refine the estimated values. Initially, the missing values are estimated based on nearby known values, and these estimations are further adjusted with a regression model, trained on temporal features (e.g. time of day, day of the week) and contextual features (temperature, humidity, wind speed).

The main goal is to produce estimations that closely reflect real data while maintaining a minimum level of mean squared error (MSE).

6.2 Problem Definition

6.2.1 Formal Statement of the Prediction Task

The main goal of the prediction model is to estimate the missing values of the pollutant factors, in this case PM_{10}, $PM_{2.5}$ and NO_2, for geographical points and temporal moments that don't have measurements directly gathered from the sensors. Formally, let a partial dataset be defined as:

$$(x, y, t, f_1, f_2, \ldots, f_n, p)$$

Where:

- x, y - represents geographic coordinates, latitude and longitude
- t - is the timestamp, providing information about date and time
- $f_1, f_2, \ldots, f_n$ - are contextual variables (temperature, humidity, pressure, wind speed, day of the week, etc.)
- p - the value of the pollutant factor

The goal of the analysis was to find a function:

$$\hat{p} = f(x, y, t, f_1, f_2, \ldots, f_n)$$

that can predict the missing values $\hat{p}$ at a given location and timestamp.

The model is a hybrid one:

1. First an estimated value is computed using IDW algorithm: $\hat{p}_{IDW}$;
2. the residuals resulted from the previous step are corrected using the XGBRegressor: $\hat{r}_{XGB}$
3. The residual correction is applied to the estimated values and the final prediction is obtained like this:

$$\hat{p}_{final} = \hat{p}_{IDW} + \hat{r}_{XGB}$$

6.2.2 Input Variables

The input variables used in prediction are divided in 3 categories:

1. Geographical coordinates:
   - latitude, longitude - needed for both interpolation and identifying the required area shown to the user;
2. Temporal factors:
   - hour_of_day, day_of_week, month: captures the cyclical influences of traffic, and temperature during the day, week and seasons;
3. Contextual factors (regarding the climate):
   - temperature, humidity and wind speed - affects the dispersion of the pollutant particles in the air;

These variables are processed, some of them are normalized and the others are transformed (e.g. timestamp is converted to day/month/hour). The XGB regressor learns the relationship between these variables and the estimated pollutant value.

6.2.3 Output Values

The model's output is represented by an array of continuous values which contains the estimated values for every targeted factor. Moreover, an important aspect is that the model uses multi-output regression[1] for predicting multiple targets at once, not just one as usual. This was possible because the predicted factors are not affecting each other.

The predicted values are:

1. $\hat{p}_{PM_{10}}$
2. $\hat{p}_{PM_{2.5}}$
3. $\hat{p}_{NO_2}$

[1] https://www.geeksforgeeks.org/machine-learning/multioutput-regression-in-machine-learning/

6.3 Data Sources and Structure

One of the most challenges in implementing this model was to successfully find a valid dataset for training. The process of data collecting got significantly more difficult since the project was in its early stages and the data required for training needed to be collected over at least one year, for identifying temporal patterns, and from 3 different stations of air monitoring (minimum points for interpolation).

The research started with the goal of getting data for the all pollutant factors that our devices are capturing (PM_{10}, $PM_{2.5}$, $PM_{1.0}$, CO_2), but a few issues were encountered:

1. CO_2 is not considered a pollutant factor [2], but it still represents a substance that is affecting the air quality. Therefore, most of stations are not collecting data for this gas.
2. $PM_{1.0}$ (particulate matter) is a subset of $PM_{2.5}$, meaning that all the particles which are $PM_{1.0}$, are also $PM_{2.5}$. In this case, the majority of stations choose to capture data only for $PM_{2.5}$, making the process of collecting more difficult.

The solution to the CO_2 problem, was to replace it with another greenhouse gas, NO_2, which is produced from burning fuel, particularly from car emissions. Since both gases are generated in similar ways, this solution represented a reasonable temporary option for training the model. Additionally, because temporal patterns are mostly influenced by traffic variations during the day, week, or season, the difference in gas type would not significantly impact the model's training. Regarding $PM_{1.0}$, the solution was to drop this pollutant due to its similarity to the $PM_{2.5}$.

After these important decisions, the new needed dataset was consisted of values for these pollutant factors: PM_{10}, $PM_{2.5}$, and NO_2.

The *European city air quality viewer*[3] website was used to fetch the necessary data. A complete dataset was obtained for the year 2022 and from the stations located in: *Piața Obor*, *Universitate*, *Crângași*. A sample of the initial dataset can be seen in the Figure 19.

start	end	pm10	pm2_5	no2	temperature	humidity	wind_speed	pressure	longitude	latitude	location
2022-01-02 00:00:00	2022-01-02 01:00:00	46.48	42.36	30.2	4.8	100.0	7.2	1021.3	26.036694	44.447275	Crangasi
2022-01-02 00:00:00	2022-01-02 01:00:00	43.33	32.42782536	44.18	4.8	100.0	7.2	1021.3	26.127289	44.444925	Piata Obor
2022-01-02 00:00:00	2022-01-02 01:00:00	49.69	37.18759848	35.06	4.8	100.0	7.2	1021.3	26.098297	44.435044	Universitate
2022-01-02 01:00:00	2022-01-02 02:00:00	33.5	30.73	18.84	3.6	100.0	3.6	1021.8	26.036694	44.447275	Crangasi
2022-01-02 01:00:00	2022-01-02 02:00:00	10.62	7.947923039999999	31.1	3.6	100.0	3.6	1021.8	26.127289	44.444925	Piata Obor
2022-01-02 01:00:00	2022-01-02 02:00:00	13.41	10.03593672	22.63	3.6	100.0	3.6	1021.8	26.098297	44.435044	Universitate
2022-01-02 02:00:00	2022-01-02 03:00:00	19.34	17.39	16.96	2.9	100.0	3.6	1021.8	26.036694	44.447275	Crangasi

Figure 19: Initial Dataset

[2]https://www.breeze-technologies.de/blog/carbon-dioxide-co2/

[3]https://www.eea.europa.eu/en/topics/in-depth/air-pollution/european-city-air-quality-viewer

6.4 Exploratory Data Analysis

During the analysis process, the main objective was to identify temporal patterns. Therefore, the dataset passed through several stages of data analysis:

1. **Data distribution:**
   The analysis of the distribution histograms (Figure 20) highlighted important characteristics of the variables within the dataset. The distributions of the pollutant factors are strongly right-skewed, indicating a high frequency of low values and the presence of some outliers in the upper range of the plots. This suggests that the majority of the measurements contain low pollution levels, but higher concentrations were captured in certain interval, most likely due to traffic or weather conditions.
   The remaining variables display uniform and bimodal distributions, which likely reflect seasonal or weather-related differences, as expected.

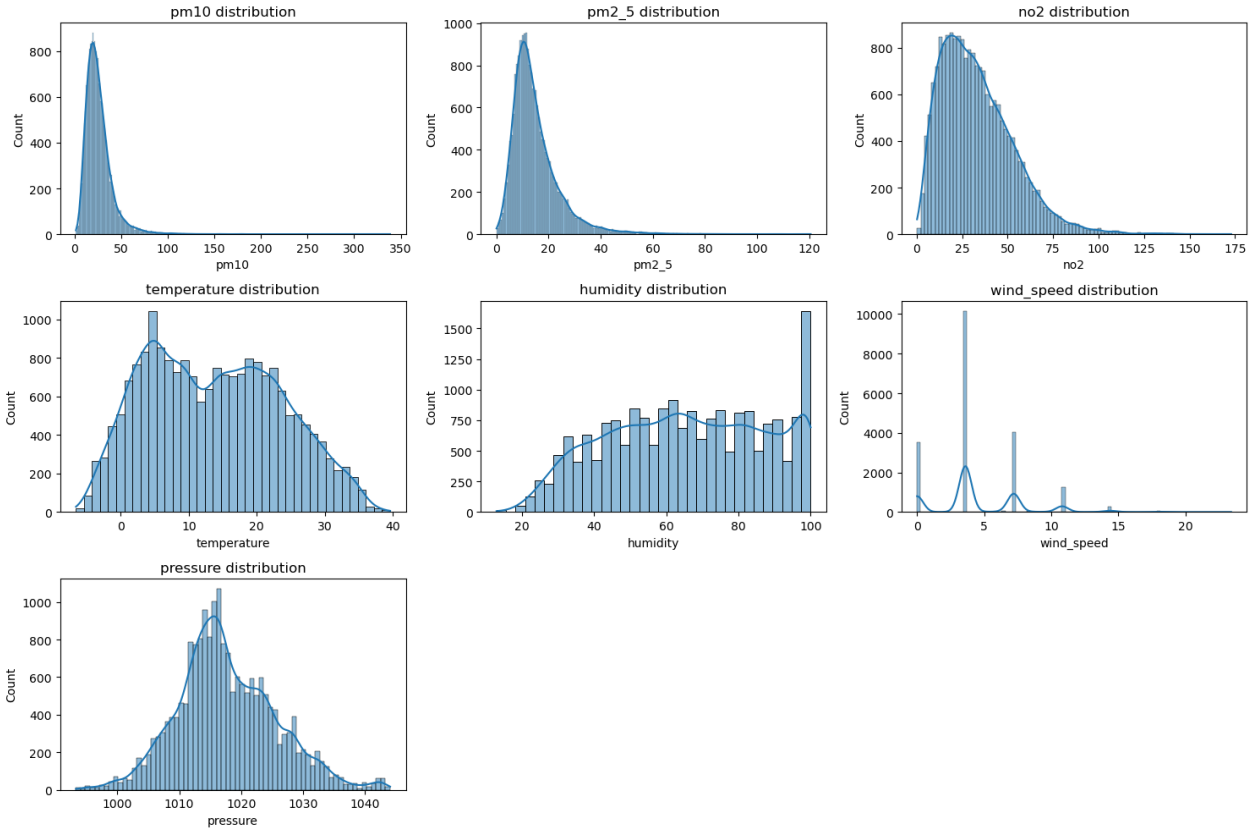


Figure 20: Data distribution

2. **Temporal trends Analysis:**
   As it was detailed before, pollution levels are directly dependent on traffic intensity. Therefore, temporal analysis was essential for identifying how pollutant concentrations fluctuate throughout the day and across different days of the week.
   In the heatmap presented below, Figure 21, areas of high concentration (highlighted in red) can be observed, varying by location depending on how heavily trafficked each area

is:

- *Crângași* - In this locations peak pollution hours are between 6:00 am - 8:00 am and 6:30 pm - 10:30 pm on weekdays. During weekends, concentrations are noticebly lower
- *Piața Obor* - A significant difference can be observed between the levels of $PM_{2.5}$ and PM_{10} compared to NO_2. While particulate matter tend to follow the same pattern as in the previous location, NO_2 concentrations are considerably higher during the day, especially between 6:00am amd 10:00pm on weekdays, most likely due to high traffic area.
- *Universitate* - The particulate matter concentrations again follow a similar pattern like the other locations, with NO_2 levels peaking during two main intervals: 6:00 am - 8:00 am and 6:30 pm - 10:30 pm on weekdays. These values decrease significantly during weekends.

The conclusions of this analysis is that all three locations contains well-defined temporal patterns that correlate with peak traffic hours and urban activity. The highest pollutant levels are consistently observed during the times when people commute to and from work.

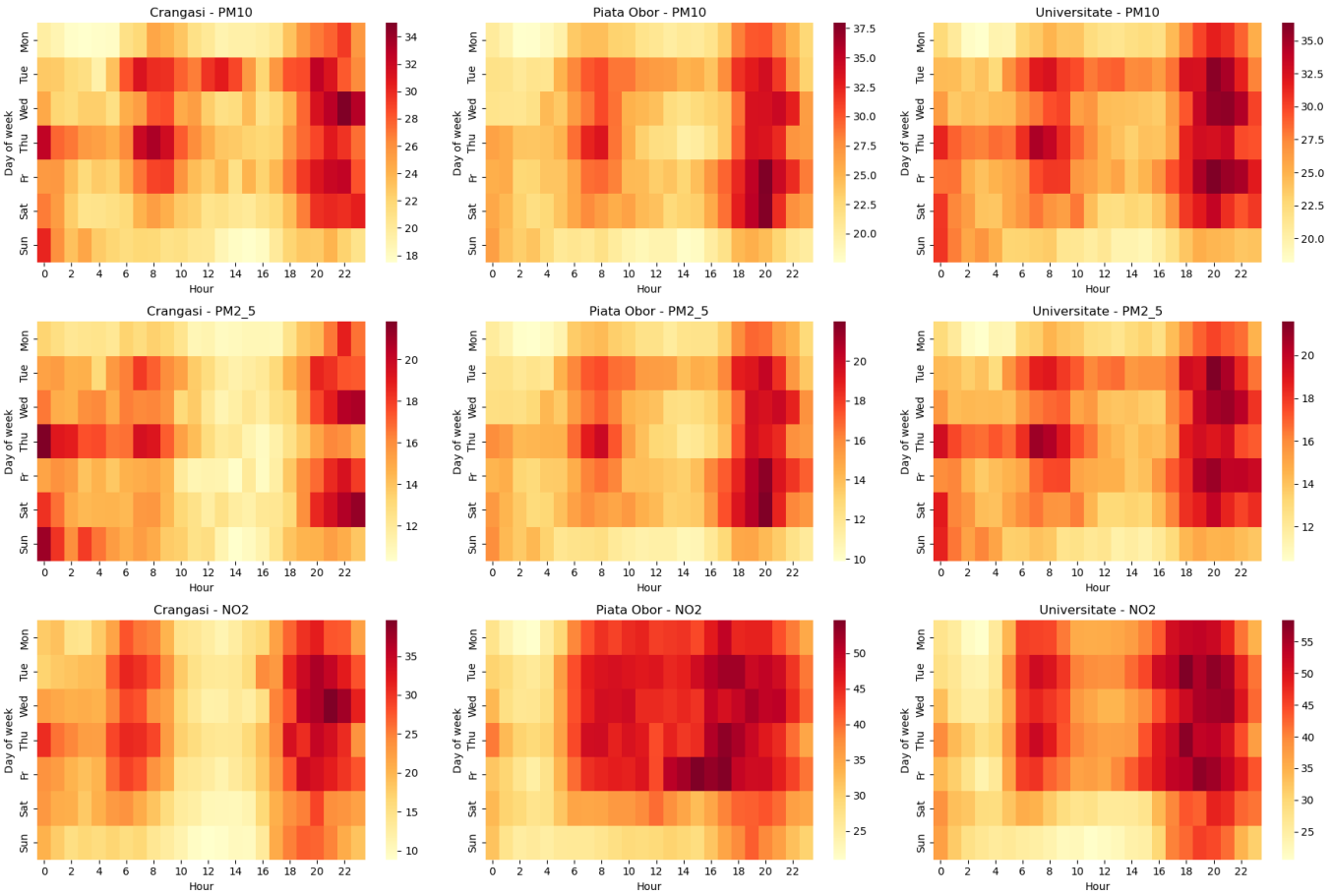


Figure 21: Daily Patterns

Additionally, plots were created to detect seasonal patterns, as shown in Figure 22. The

following conclusions were observed:

- $PM_{2.5}$ **and** PM_{10} - These pollutants show significant temporal variation with frequent peaks. During autumn and spring, values exceeded over 50–300 g/m³ in Crângași, significantly higher than the other locations. Universitate and Piața Obor present moderate pollution levels, but with visible peaks during the same periods, indicating a possible seasonal influence.
- NO_2 - Concentrations are more elevated compared to the other pollutants, with a gradual increase observed in the second half of the year(September to December), likely due to the restart of urban activities after the summer period.

In conclusion, all pollutant factors have noticeable seasonal oscillations, with peaks during spring and late autumn. Crângași appears to be the location with the highest particulate matter levels, while Piața Obor shows elevated values of NO_2, suggesting a region with an intense traffic. Universitate has an intermediate behavior, but with higher NO_2 levels during the latter part of the year.

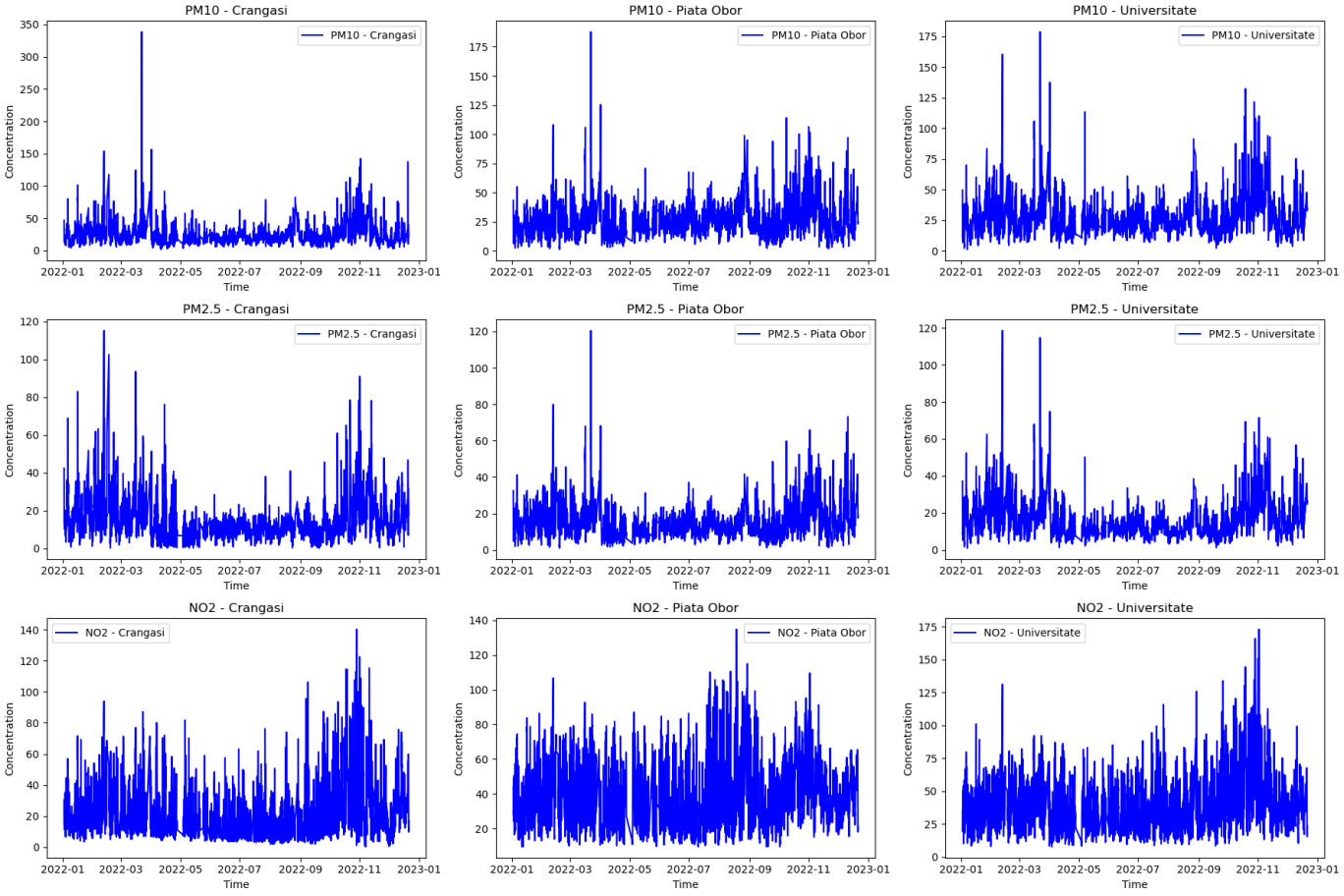


Figure 22: Seasonal Patterns

3. **Correlation Heatmap**
   Observing the correlation between variables was essential for training an effective model. In Figure 23 a strong correlation (0.88) can be seen between the particulate matter values, as expected. NO_2 shows an interesting correlation with longitude (0.4), likely due

to specific high-traffic areas where fuel emissions lead to significantly higher concentrations of NO_2. A negative correlation is noticed between temperature and the particulate matter levels, indicating that PM concentrations tend to be lower on warmer days, most likely due to more efficient atmospheric dispersion.

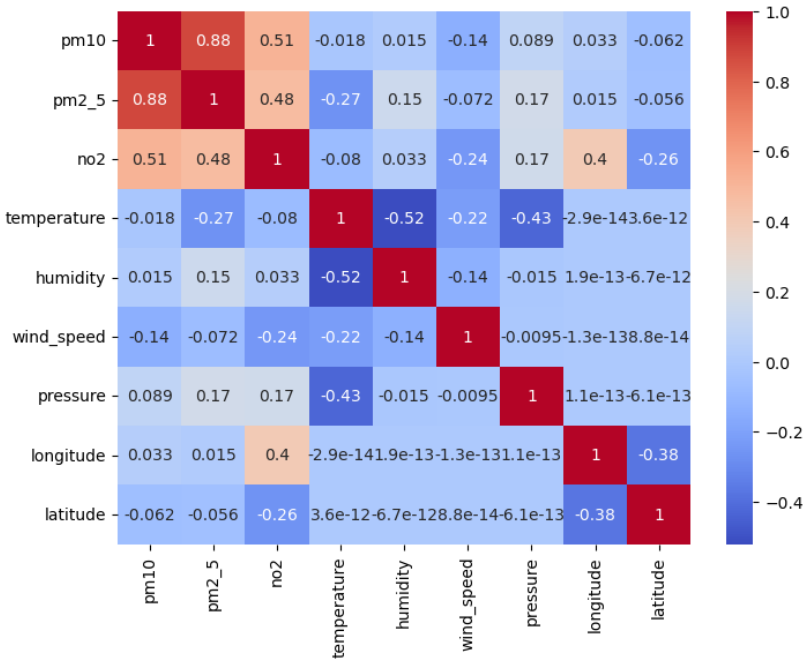


Figure 23: Correlations

6.5 Implementation Details

6.5.1 Interpolation using IDW

Interpolation is a method for finding new data points based on a discrete set of known points. *IDW* stands for *Inverse Distance Weighting* [4], a deterministic interpolation algorithm used frequently in spatial analysis, mostly when data is available for only a limited number of geographic locations.

The main principle behind this algorithm is that the influence of a reference point on the estimated value decreases as the distance increases[23]. Moreover, the points located nearby the targeted location have a greater influence on the estimated value, than those located farther away. In more mathematical terms, the algorithm estimates a value z at a given location, based on a weight x, representing the mean of the nearby observations, defined like in equation 2.

[4]https://www.geo.fu-berlin.de/en/v/soga-r/Advances-statistics/Geostatistics/Inverse-Distance-Weighting-IDW/index.html

$$\hat{z}(x) = \frac{\sum_{i=1}^{N} w_i \cdot z(x_i)}{\sum_{i=1}^{N} w_i} \tag{2}$$

where the weights are defined by this formula presented in equation 3

$$w_i = \frac{1}{d(x, x_i)^p} \tag{3}$$

And also having:

- $\hat{z}(x)$ - value estimated for an unknown location, x
- $z(x_i)$ - known value for the location (x_i)
- $d(x, x_i)$ - euclidian distance between the known location and the unknown one
- p - determines the degree influence from the nearby points, compared to those farther away (it is usually 1 or 2)
- N - the total number of points used in interpolation

In this project, the IDW algorithm was used to estimate pollutant factors at locations that don't have direct measurements from the monitoring devices.

6.5.2 XGBoost Regressor

XGBoost (*Extreme Gradient Boosting*[5]) is a machine learning algorithm that relies on decision trees and is widely appreciated for its strong performance in both regression and classification tasks. It builds upon the traditional gradient boosting approach, offering improvements in speed, scalability, and control over overfitting.

The model is constructed in a step-by-step manner, where each new tree is trained to minimize the errors made by the previous ensemble. The final output is a weighted combination of all the trees created throughout the process, allowing for refined and accurate predictions.

Some of the key strengths of XGBoost include:

- The use of both L1 and L2 regularization techniques to reduce the risk of overfitting;
- Built-in support for handling missing data;
- Compatibility with parallel computation and GPU acceleration;
- The ability to learn complex, non-linear relationships between input features.

6.5.3 Final model: Spatial Interpolation and Correction using XGB

The model's implementation followed a hybrid approach, organized into two different stages, each with a clear role in improving prediction accuracy (Figure 24).

[5]https://www.geeksforgeeks.org/machine-learning/xgboost-for-regression/

The model was trained using pairs of datasets to predict the third location. For example, datasets from *Piața Obor* and *Universitate* were used to predict data for *Crângași*, and datasets from *Crângași* and *Universitate* were used to predict data for *Piața Obor*. Once the regressor was trained, the evaluation was performed using *Universitate* as the prediction target.

As previously mentioned, the model consists of two prediction stages:

1. The first stage involves estimating the initial values of the pollutant factors (PM_{10}, $PM_{2.5}$, NO_2), using the spatial interpolation algorithm IDW. This method was chosen for its simplicity and its ability to quickly estimate values for locations lacking direct measurements.
2. In the second stage, a regression model, based on XGBoost algorithm was applied to predict the residuals the difference between the real values and those estimated by IDW. This strategy allows the model to learn the relationship between these residuals and various input variables: weather-related features (temperature, humidity, wind speed, pressure), temporal factors (hour, day of the week, month) and geographic coordinates(latitude and longitude). The final predicted values are obtained by adding the residuals estimated by the regressor to the IDW-calculated values from the first stage.

Additionally for a robust and well-calibrated version of the model the following techniques were integrated:

- **MultiOutputRegressor**[6] - used to enable the simultaneous prediction of all three pollutant factors, allowing independently training of the model for each target. In other words, the model was able to output multiple predicts in a single iteration.
- **GridSearch**[7] - integrated to optimize the hyperparameters(e.g. number of trees, the maximum depth or learning rate), improving performance.

6.6 Model Evaluation - Results and Analysis

For evaluating the performance of the hybrid model, three relevant metrics were calculated for each pollutant: the Mean Absolute Error (MAE), the Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2). These metrics provide an overview of the model's ability to capture variations in the temporal data.

The metrics are designed to capture the following:

- **MAE** - the mean absolute error between real values and those estimated by the model. The smaller the MAE, the closer to reality are the predictions

[6]https://scikit-learn.org/stable/modules/generated/sklearn.multioutput.MultiOutputRegressor.html
[7]https://www.dremio.com/wiki/grid-search/

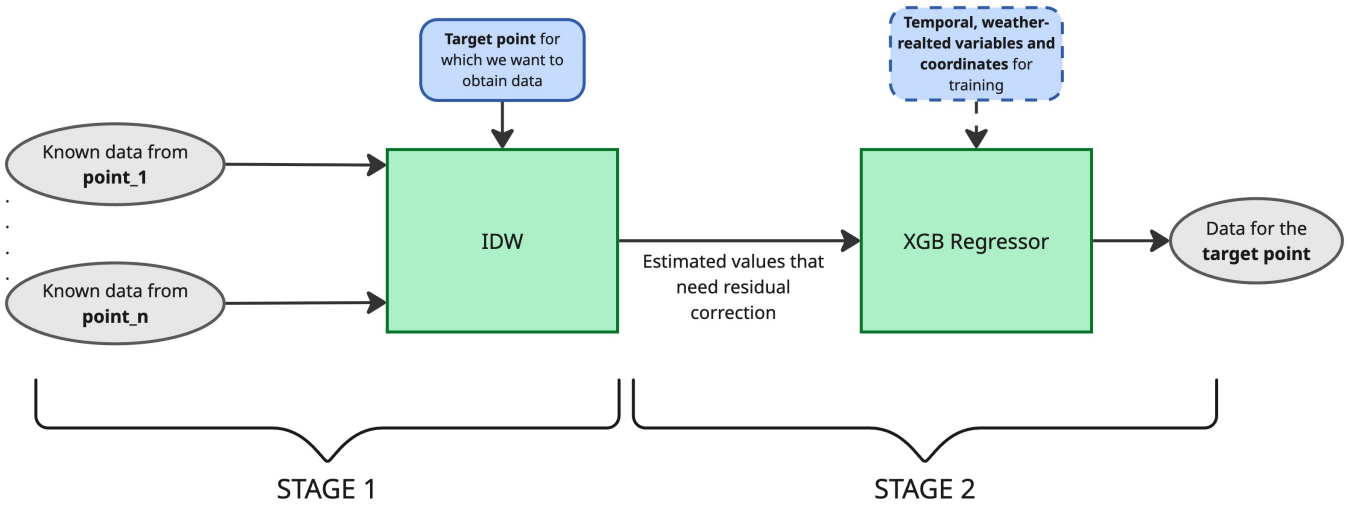


Figure 24: High-level diagram

- **RMSE** - measures the standard deviation of the residuals, having a larger penalty for bigger errors. It is useful for identifying how well the model avoids large deviations.
- **R^2** - a score closer to 1 indicates that the model provides a good fit, while a score closer to 0 suggests that the model fails to capture the variability in the data.

The results of the performance metrics are:

1. **NO_2:**
   - *MAE*: 7.05 µg/m^3
   - *RMSE*: 10.15 µg/m^3
   - R^2: 0.713
2. **PM_{10}:**
   - *MAE*: 5.35 µg/m^3
   - *RMSE*: 8.25 µg/m^3
   - R^2: 0.712
3. **$PM_{2.5}$:**
   - *MAE*: 3.41 µg/m^3
   - *RMSE*: 5.15 µg/m^3
   - R^2: 0.713

The results obtained are promising. For PM_{10}, the model got a Mean Absolute Error of 5.35 and a R^2 score of 0.712, meaning that approximately 71% of the real-time variations can be explained by the model. Although, $PM_{2.5}$ has lower MAE and RMSE values (3.41 and 5.15), but the R^2 score is slightly lower (0.667), suggesting a moderate predictive capability. NO_2 obtained the highest RMSE (10.15), but also the highest R^2 score (0.713), indicating that

the model manages to capture general trends well, despite the larger variability in the actual values.

The evaluation graphs demonstrates a strong performance for the predictive model across all pollutants. The predicted values, represented by the orange lines, closely track and largely overlap with the true values, represented by the blue lines. This indicates that the model effectively captures the general trends and fluctuations from the real-world data.

As it is in Figure 25 and Figure 26, the real values for PM_{10} and $PM_{2.5}$ generally present moderate variations, with notable outliers occurring in February, March, and November. It is observed in both graphs that the initial significant outlier (a large spike in February) was not predicted by the model. However, by March, the model shows improvement, with predictions for the following spikes being much closer to the true values. This suggests a clear learning progression within the model, where it adapts and better captures the patterns after the exposure to extreme values.

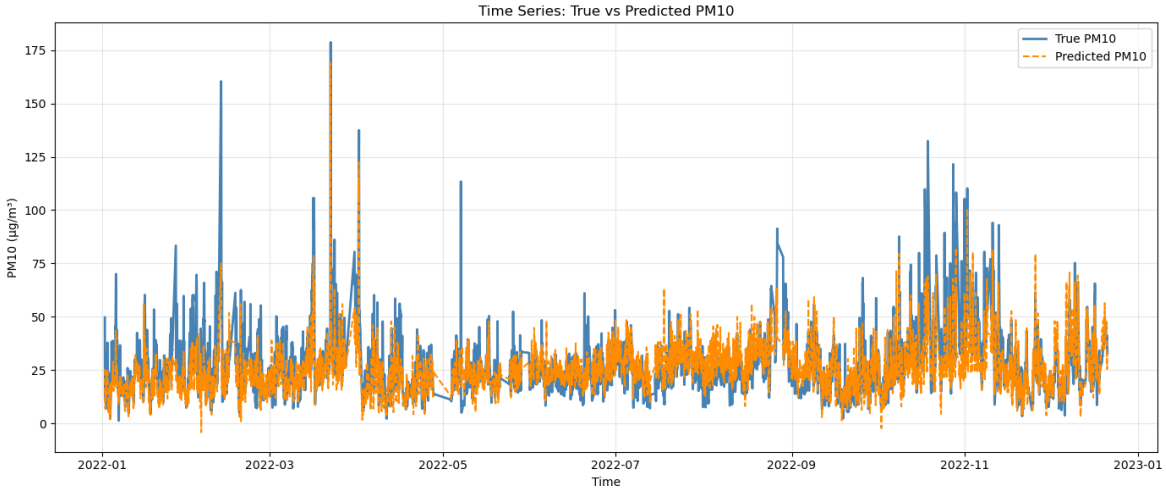


Figure 25: Real and predicted values for PM_{10}

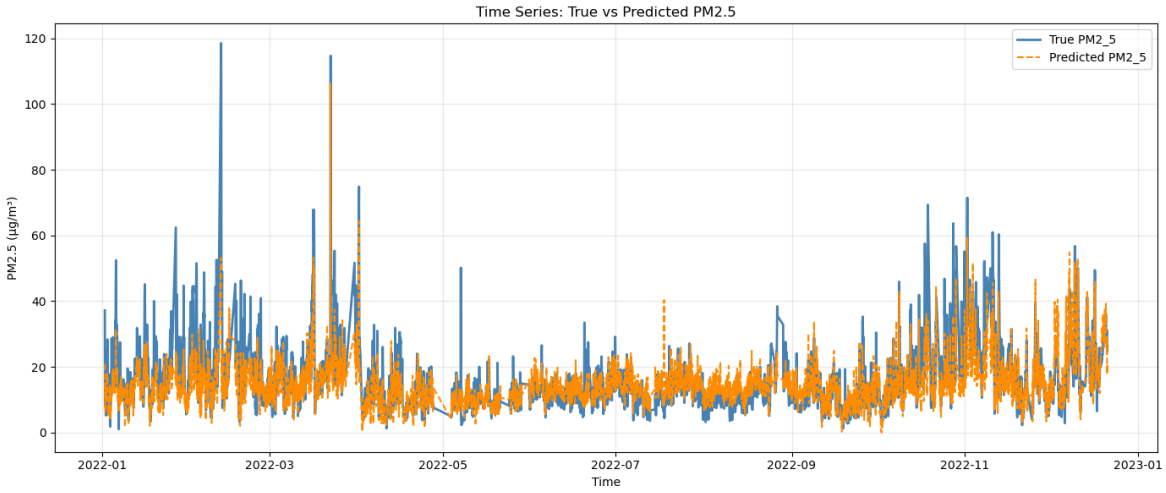


Figure 26: Real and predicted values for $PM_{2.5}$

In the case of NO_2, Figure 27, it is seen that the general trends were captured well by the model, but there is a region during the months of October and November, when you can see that the model didn't succeed in predicting values close to the real ones, probably because of the big data deviation.

For NO_2, as shown in Figure 27, the model generally succeeds in capturing the overall trends. However, a specific region during the months of October and November reveals a limitation: the model's predictions deviate more from the real values. This decreased accuracy in this particular period is likely because of unexpected and long period of extreme values.

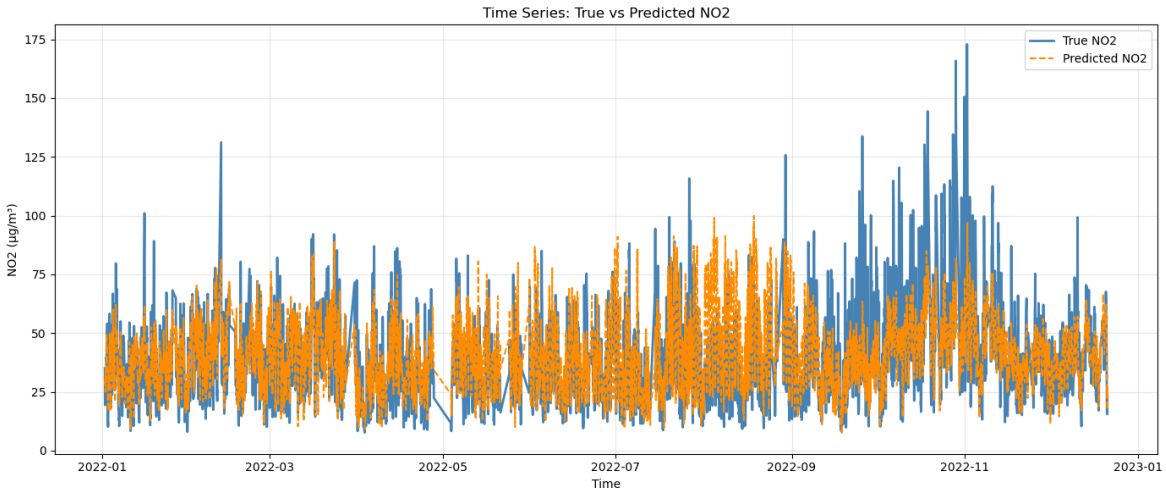


Figure 27: Real and predicted values for NO_2

In conclusion, the model demonstrates a strong ability to predict general trends, with further potential for improvement in forecasting specific anomaly events throughout the year.

7 CONCLUSIONS AND FURTHER WORK

7.1 Conclusions

As detailed throughout the previous chapters, this project set out to design and implement an application for air quality monitoring and prediction in urban regions, having integrated modern artificial intelligence techniques and a robust backend infrastructure. After completing processes of research, development, and testing, I can confirm that the initial objectives have been fully achieved. My personal contribution is reflected in several key accomplishments:

- **Complete infrastructure development:** I developed the backend system from scratch, ensuring seamless integration of all necessary technologies. The architecture was carefully designed to be scalable, maintainable, and reliable, being capable of supporting future feature expansion. As a result, the system is not only functional but also optimized for performance and long-term sustainability.
- **Prediction model for pollutant levels:** The project brought valuable contributions in the field of AI by integrating a model that improves data consistency. After a research phase, I combined an interpolation algorithm with a XGBoost Regressor to increase the prediction accuracy. Despite not being trained on real-time data, the model demonstrated a promising level of accuracy for an off-site solution, showing that it is also prepared for backend integration.
- **Technical and functional validation of the system:** The entire system was carefully tested in different situations to check how fast it responds, how it behaves, and how accurate it is. Additionally, performance metrics were collected using different platforms, and the results showed that the system works well and meets all the goals of the project.

In conclusion, my contribution represents a solid step forward in the development of CityAirQ project. The infrastructure I built provides a strong and flexible foundation, ready to support advanced data analysis and new functionalities as the platform evolves. Overall, this project highlights my ability to combine research, software engineering, and data science into a complete, functioning solution with real-world relevance.

7.2 Further Work

There are some aspects of the project that can be improved. First, on the backend side, an optimized communication mechanism between the client and the server should be implemented to significantly reduce latency. Another important improvement would be the integration of

the AI model into the infrastructure, along with the establishment of a system for managing and monitoring it. This system should support continuous evaluation, trigger-based retraining, and the collection of metrics to track the model's performance over time. Additionally, the model needs to be properly deployed, and a data orchestration system should be implemented to split data into training and evaluation sets. Once the model achieves a good performance, a new communication layer should be introduced to interact with the database service, allowing the model to automatically fill in missing entries and improve data completeness.

BIBLIOGRAPHY

[1] Hannah Ritchie and Max Roser. Air pollution. *Our World in Data*, 2021. Available at: https://ourworldindata.org/air-pollution.

[2] Wikipedia. Air quality index. [Online]. Available: `https://en.wikipedia.org/wiki/Air_quality_index`, 2025. [Accessed: 30 May 2025].

[3] David Fowler, Peter Brimblecombe, John Burrows, Mathew R. Heal, Peringe Grennfelt, David S. Stevenson, Alan Jowett, Eiko Nemitz, Mhairi Coyle, Xuejun Liu, Yunhua Chang, Gary W. Fuller, Mark A. Sutton, Zbigniew Klimont, Mike H. Unsworth, and Massimo Vieno. A chronology of global air quality. *Philosophical Transactions of the Royal Society A*, 378(2183):20190314, 2020.

[4] Massive Earth Foundation. The dark side of industrial revolution: Environmental destruction. [Online]. Available: `https://massivefoundation.org/opinion/dark-side-of-industrial-revolution/`, 2024. [Accessed: May 26, 2025].

[5] Victor Stoica. A brief analysis of the sustainable mobility approach in bucharest. *ResearchGate*, 2016. Available at: `https://www.researchgate.net/publication/299405793_A_Brief_Analysis_of_the_Sustainable_Mobility_Approach_in_Bucharest`.

[6] L. L. Popescu, R. S. Popescu, and T. Catalina. Indoor particle's pollution in bucharest, romania. *Toxics*, 10(12):757, December 2022.

[7] Soulmatesventures. The story behind green technology. [Online]. Available: `https://www.soulmatesventures.com/the-story-behind-green-technology/`, 2025. [Accessed: 28 May 2025].

[8] Airly. The role of technology in reducing air pollution. [Online]. Available: `https://airly.org/en/the-role-of-technology-in-reducing-air-pollution/`, 2025. [Accessed: 28 May 2025].

[9] Clarity. Nitrogen dioxide health. [Online]. Available: `https://www.clarity.io/blog/how-do-air-quality-monitors-work`, 2025. [Accessed: 28 May 2025].

[10] California Air resources Board. Nitrogen dioxide health. [Online]. Available: `https://ww2.arb.ca.gov/resources/nitrogen-dioxide-and-health`, 2025. [Accessed: 28 May 2025].

[11] American Lung Association. Sulfur dioxide. [Online]. Available: https://www.lung.org/clean-air/outdoors/what-makes-air-unhealthy/sulfur-dioxide, 2025. [Accessed: 28 May 2025].

[12] California Air resources Board. Carbon monoxide health. [Online]. Available: https://ww2.arb.ca.gov/resources/carbon-monoxide-and-health, 2025. [Accessed: 28 May 2025].

[13] Prana Air. What is air quality index? [Online]. Available: https://www.pranaair.com/blog/what-is-air-quality-index-aqi-and-its-calculation/, 2025. [Accessed: 29 May 2025].

[14] Claudia Banciu, Adrian Florea, and Razvan Bogdan. Monitoring and predicting air quality with iot devices. *Processes*, 12:1961, 09 2024.

[15] Montaser N.A. Ramadan, Mohammed A.H. Ali, Shin Yee Khoo, Mohammad Alkhedher, and Mohammad Alherbawi. Real-time iot-powered ai system for monitoring and forecasting of air pollution in industrial environment. *Ecotoxicology and Environmental Safety*, 283:116856, 2024.

[16] BigDataWire. Understanding open data architecture and time series data. [Online]. Available: https://www.bigdatawire.com/2023/10/19/understanding-open-data-architecture-and-time-series-data/, 2025. [Accessed: 31 May 2025].

[17] TechTarget. What is scalability? [Online]. Available: https://www.techtarget.com/searchdatacenter/definition/scalability, 2025. [Accessed: 1 June 2025].

[18] NordVPN. System security. [Online]. Available: https://nordvpn.com/cybersecurity/glossary/system-security/, 2025. [Accessed: 1 June 2025].

[19] Geeks For Geeks. System security. [Online]. Available: https://www.geeksforgeeks.org/client-server-architecture-system-design/, 2025. [Accessed: 1 June 2025].

[20] Geeks For Geeks. Communication protocols in system design. [Online]. Available: https://www.geeksforgeeks.org/communication-protocols-in-system-design/, 2025. [Accessed: 1 June 2025].

[21] MuleSoft. Top 3 benefits of rest apis. [Online]. Available: https://www.mulesoft.com/api/rest/top-3-benefits-of-rest-apis, 2025. [Accessed: 3 June 2025].

[22] Freie Universitat Berlin. Inverse distance weighting (idw). [Online]. Available: https://www.geo.fu-berlin.de/en/v/soga-r/Advances-statistics/Geostatistics/Inverse-Distance-Weighting-IDW/index.html, 2025. [Accessed: 12 June 2025].

[23] George Y. Lu and David W. Wong. An adaptive inverse-distance weighting spatial interpolation technique. *Computers Geosciences*, 34(9):1044–1055, 2008.